

國立中央大學

資訊工程學系  
碩士論文

基於 SQL 查詢負載評估的  
Presto auto-scaling 機制

**Presto auto-scaling mechanism based on SQL  
query workload estimation**

研究生：林彥豪

指導教授：王尉任 博士

中華民國 一百一十四 年 六 月

# 國立中央大學圖書館學位論文授權書

填單日期：114 / 8 / 11

2019.9 版

|       |                                       |      |                                                                    |
|-------|---------------------------------------|------|--------------------------------------------------------------------|
| 授權人姓名 | 林彥豪                                   | 學號   | 112552014                                                          |
| 系所名稱  | 資訊電機學院資訊工程學系 - 不分組                    | 學位類別 | <input checked="" type="checkbox"/> 碩士 <input type="checkbox"/> 博士 |
| 論文名稱  | 基於 SQL 查詢負載評估的 Presto Auto-scaling 機制 | 指導教授 | 王尉任                                                                |

## 學位論文網路公開授權

授權本人撰寫之學位論文全文電子檔：

• 在「國立中央大學圖書館博碩士論文系統」

(☒)同意立即網路公開

( )同意 於西元\_\_\_\_\_年\_\_\_\_\_月\_\_\_\_\_日網路公開

( )不同意網路公開，原因是：\_\_\_\_\_

• 在國家圖書館「臺灣博碩士論文知識加值系統」

(☒)同意立即網路公開

( )同意 於西元\_\_\_\_\_年\_\_\_\_\_月\_\_\_\_\_日網路公開

( )不同意網路公開，原因是：\_\_\_\_\_

依著作權法規定，非專屬、無償授權國立中央大學、台灣聯合大學系統與國家圖書館，不限地域、時間與次數，以文件、錄影帶、錄音帶、光碟、微縮、數位化或其他方式將上列授權標的基於非營利目的進行重製。

## 學位論文紙本延後公開申請 (紙本學位論文立即公開者此欄免填)

本人撰寫之學位論文紙本因以下原因將延後公開

• 延後原因

( )已申請專利並檢附證明，專利申請案號：

( )準備以上列論文投稿期刊

( )涉國家機密

( )依法不得提供，請說明：\_\_\_\_\_

• 公開日期：西元\_\_\_\_\_年\_\_\_\_\_月\_\_\_\_\_日

※繳交教務處註冊組之紙本論文(送繳國家圖書館)若不立即公開，請加填「國家圖書館學位論文延後公開申請書」

研究生簽名：林彥豪

指導教授簽名：王尉任

\*本授權書請完整填寫並親筆簽名後，裝訂於論文封面之次頁。

## 摘要

隨著大數據時代數據量快速增長，對於高效數據處理技術的需求日益迫切。

Massively Parallel Processing (MPP) 技術作為處理大規模數據查詢的架構，其中開源的 Presto SQL 查詢引擎因其高效能而廣受關注。然而 Presto 原生的靜態資源配置策略難以適應動態變化的查詢負載，會導致資源利用率低或不必要的浪費，進而影響查詢效能與成本效益。為解決此問題，本研究提出一種基於 SQL 查詢語句進行負載評估的 Presto Auto Scaling 機制。該機制整合了 TF-IDF 向量化進行 SQL 語句的特徵提取，並利用 XGBoost 建立分類模型來預測資源的使用範圍，結合基礎設施即代碼 (Infrastructure as Code) 技術與 Apache Airflow 自動化工作流程，實現 Presto 工作節點的調整。本研究在 Oracle Cloud Infrastructure 環境中，使用 TPC-DS Benchmark 測試集對該機制進行了驗證。實驗結果表明，相較於一個 Coordinator 搭配三個 Worker 的配置，本研究的 Auto Scaling 機制使 99 筆 TPC-DS 查詢的總執行時間減少了 14%，並且 92% 的查詢延遲下降，其中聚合查詢的查詢延遲降低了 20%。且得益於查詢執行的時間降低，整體的成本效益提升將近 5%，然而整體雲端運算成本增加了 12%，主要因平均節點數從 4 個增加至 5.32 個所致。儘管查詢效能得到優化，實驗亦觀察到 69% 的查詢記憶體使用量有所上升，平均增加 15-20%，間接導致查詢執行時間未達預期低值；初步分析可能歸因於在增加 Presto Worker 之後，資料在 Worker 之間交互傳遞所造成的影響。這些結果顯示在動態資源配置中，實現查詢效能與成本控制之間的權衡仍面臨挑戰。

關鍵字：Presto、Auto Scaling、機器學習、資源管理、雲端成本

# Abstract

With the rapid growth of data in the era of big data, the demand for efficient data processing technologies has become increasingly urgent. Massively Parallel Processing (MPP) technology has emerged as a key architecture for handling large-scale data queries, and among MPP solutions, the open-source Presto SQL query engine has gained wide attention for its high performance. However, Presto's native static resource allocation strategy struggles to adapt to dynamically changing query workloads, often leading to low resource utilization or unnecessary waste, which in turn affects query performance and cost-efficiency. To address this issue, this study proposes a Presto Auto Scaling mechanism based on workload assessment through SQL queries. The mechanism integrates TF-IDF vectorization for SQL feature extraction and leverages an XGBoost-based classification model to predict the required resource range. It also combines Infrastructure as Code (IaC) practices and Apache Airflow automation workflows to dynamically adjust the number of Presto worker nodes. The proposed mechanism was validated in an Oracle Cloud Infrastructure environment using the TPC-DS benchmark dataset. Experimental results show that, compared to a static configuration with one coordinator and three workers, the Auto Scaling mechanism reduced the total execution time of 99 TPC-DS queries by 14%, with 92% of the queries experiencing reduced latency. In particular, the latency of aggregate queries decreased by 20%. Additionally, due to the shortened execution times, overall cost-efficiency improved by nearly 5%. However, total cloud computing costs increased by 12%, mainly because the average number of worker nodes rose from 4 to 5.32. Despite the improvements in query performance, the experiments also observed increased memory usage in 69% of the queries, with an average increase of 15–20%, which indirectly prevented execution times from reaching their expected lows. Preliminary analysis suggests that this may be due to the increased data exchange overhead among worker nodes after scaling out. These results indicate that in dynamic resource allocation scenarios, achieving a balance between query performance and cost control remains a challenging task.

# 目錄

|                                         |     |
|-----------------------------------------|-----|
| 摘要 .....                                | i   |
| Abstract .....                          | ii  |
| 目錄 .....                                | iii |
| 圖目錄 .....                               | v   |
| 表目錄 .....                               | vii |
| 一、 緒論 .....                             | 1   |
| 1-1 研究背景 .....                          | 1   |
| 1-2 研究動機與目的 .....                       | 2   |
| 1-3 問題定義與預期貢獻 .....                     | 3   |
| 1-4 論文架構 .....                          | 4   |
| 二、 背景知識 .....                           | 5   |
| 2-1 Massively Parallel Processing ..... | 5   |
| 2-2 Presto .....                        | 7   |
| 2-3 Apache Airflow .....                | 9   |
| 2-4 Terraform .....                     | 11  |
| 三、 相關研究 .....                           | 13  |
| 3-1 Presto 分散式運算應用相關研究 .....            | 13  |
| 3-2 Auto Scaling 應用相關研究 .....           | 13  |

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 3-3 | 查詢語法分析與資源需求預測相關研究 .....                             | 14 |
| 四、  | 系統設計 .....                                          | 15 |
| 4-1 | 系統架構.....                                           | 15 |
| 4-2 | Resource Estimation Service Component.....          | 17 |
| 4-3 | Query Prediction Service Component .....            | 19 |
| 4-4 | Infrastructure Provisioning Service Component ..... | 32 |
| 4-5 | Workflow Management System .....                    | 35 |
| 五、  | 研究結果 .....                                          | 39 |
| 5-1 | 實驗環境說明.....                                         | 39 |
| 5-2 | 測試場景與對比方法.....                                      | 40 |
| 5-3 | 查詢延遲比較.....                                         | 41 |
| 5-4 | 資源使用比較.....                                         | 45 |
| 5-5 | 成本效益比較.....                                         | 50 |
| 六、  | 結論及未來研究方向 .....                                     | 56 |
| 七、  | 參考文獻.....                                           | 58 |

## 圖目錄

|                                          |    |
|------------------------------------------|----|
| 圖 1：MPP 概念圖 .....                        | 6  |
| 圖 2：Presto 架構圖 .....                     | 7  |
| 圖 3：Airflow 架構圖 .....                    | 10 |
| 圖 4：Terraform 架構圖 .....                  | 11 |
| 圖 5：Terraform 核心工作流程圖 .....              | 12 |
| 圖 6：系統架構圖 .....                          | 16 |
| 圖 7：CPU Time 分佈直方圖 .....                 | 23 |
| 圖 8：Memory 分佈直方圖 .....                   | 24 |
| 圖 9：CPU Time 與 Memory 的散點圖 .....         | 25 |
| 圖 10：SQL 語法特徵數量分佈直方圖 .....               | 26 |
| 圖 11：模型訓練的參數 .....                       | 28 |
| 圖 12：IPSC 決策邏輯流程圖 .....                  | 33 |
| 圖 13：控制 OCI VM Instance 的啟動/停止代碼示例 ..... | 34 |
| 圖 14：獲取 OCI VM Instance 的狀態代碼示例 .....    | 34 |
| 圖 15：Terraform 資源聲明代碼示例 .....            | 35 |
| 圖 16：DAG 任務一代碼示例 .....                   | 36 |
| 圖 17：DAG 任務二代碼示例 .....                   | 37 |
| 圖 18：DAG 任務三代碼示例 .....                   | 37 |

|                                           |    |
|-------------------------------------------|----|
| 圖 19：Auto Scaling 機制下各類查詢類型執行時間降低比例 ..... | 42 |
| 圖 20：Auto Scaling 機制下各類查詢執行時間數量比較 .....   | 43 |
| 圖 21：連接查詢下 SQL 語句執行時間對比 .....             | 43 |
| 圖 22：子查詢下 SQL 語句執行時間對比 .....              | 44 |
| 圖 23：窗口函數查詢下 SQL 語句執行時間對比 .....           | 44 |
| 圖 24：聚合查詢下 SQL 語句執行時間對比 .....             | 45 |
| 圖 25：Auto Scaling 後分群 Worker 數量分布 .....   | 46 |
| 圖 26: 靜態配置與 Auto Scaling 綜合比較.....        | 52 |
| 圖 27：成本效益分佈圖 .....                        | 53 |



## 表目錄

|                                               |    |
|-----------------------------------------------|----|
| 表 1：Presto 主要元件 .....                         | 8  |
| 表 2：Airflow 主要元件 .....                        | 10 |
| 表 3：Terraform 元件 .....                        | 12 |
| 表 4：系統元件 .....                                | 16 |
| 表 5：CPU Time 區間對應 Parallelism .....           | 18 |
| 表 6：Memory 區間對應代表值 .....                      | 18 |
| 表 7：資料預處理後的資料欄位 .....                         | 21 |
| 表 8：CPU Time 預測模型的每個分類的準確率以及召回率 .....         | 29 |
| 表 9：Memory 預測模型的每個分類的準確率以及召回率 .....           | 29 |
| 表 10：TPC-DS 資料集 .....                         | 39 |
| 表 11：靜態配置與 Auto Scaling 機制的總執行時間比較 .....      | 41 |
| 表 12：靜態配置與 Auto Scaling 機制下查詢分群的總執行時間比較 ..... | 42 |
| 表 13：Worker 數量變化比較 .....                      | 46 |
| 表 14：分群 Worker 數量比較 .....                     | 46 |
| 表 15：Worker 數量與 CPU Time 的比較 .....            | 47 |
| 表 16：Query 14 與 Query 39 語句分析 .....           | 47 |
| 表 17：Query 22 與 Query 72 語句分析 .....           | 48 |
| 表 18：Worker 數量與總記憶體峰值的比較 .....                | 48 |

|                                    |    |
|------------------------------------|----|
| 表 19：Query 39 與 Query95 語句分析.....  | 48 |
| 表 20：Query 72 與 Query 99 語句分析..... | 49 |
| 表 21：成本效益提升 Top 5 查詢及其效能變化.....    | 54 |
| 表 22：成本效益下降 Top 5 查詢及其效能變化.....    | 54 |

# 一、緒論

## 1-1 研究背景

隨著大數據時代的來臨，全球數據規模以驚人的速度增長。根據國際數據公司（IDC）在 2024 年的報告，預計到 2028 年，全球數據量將達到 393.8 ZB [1]。這一數據量的爆炸性增長對數據處理技術提出了更高的要求，傳統的單機處理架構已無法應對如此龐大的數據規模和複雜的查詢需求。因此，高效且可擴展的數據處理技術成為當前研究的熱點。在這一背景下，Massively Parallel Processing（MPP）技術應運而生。MPP 是一種分佈式運算架構，通過將大規模數據集分片並分配至多個運算節點，實現並行處理，從而大幅縮短查詢的運算時間。MPP 架構的優勢在於其能夠有效利用叢集中的多台機器資源，同時處理大量數據，適用於大規模數據的查詢和分析任務。

Presto 作為一款開源的 MPP SQL 查詢引擎 [1]，近年來在企業中得到了廣泛應用，在交互式查詢分析領域表現突出。Presto 支持多種數據源的查詢，並能夠在不移動數據的情況下進行數據處理。然而 Presto 在效能上具有優勢，但在面對動態變化的查詢負載時，其無法自動擴展或縮減運算資源，這導致了資源分配不足或浪費的問題；當查詢負載增加時，Presto 叢集可能因資源不足而無法及時響應；而在查詢負載減少時，過多的閒置資源則會造成不必要的成本開銷。

Presto 在企業中的重要性佔有一席之地。根據雲供應商的官方文件 [2]，Meta 的 Presto 由超過一千名員工使用，每天執行超過 30,000 筆查詢，並處理高達 1 PB（Petabyte）的數據。同樣，Netflix 每天在其 Presto 叢集上執行大約 3,500 筆查詢。這些數據顯示，Presto 已成為企業大數據分析的關鍵工具，其查詢效能的穩定性直接影響著企業的運營效率和決策速度。因此，研究如何最佳化 Presto 的資源管理，提升其在動態負載下的效能和資源利用效率，具有重要的意義和研究價值。

儘管 Presto 作為高效能分散式查詢引擎，適用於大規模數據查詢與分析，但當前 Presto 在雲端環境的資源配置仍多依賴靜態設定，導致計算資源利用效率不佳。尤其

在 Oracle Cloud Infrastructure (OCI) 等雲端環境中，運算資源的彈性調度對於控制成本與提升效能至關重要。

## 1-2 研究動機與目的

在雲端運算日益普及的背景下，像是 Oracle Cloud Infrastructure (OCI) 這樣資源有限的環境中，如何提高運算資源的使用效率成為了挑戰。Presto 作為一種 Massively Parallel Processing (MPP) 的 SQL 查詢引擎，因其能夠即時處理大規模數據集而備受青睞。然而，Presto 的資源配置為靜態，這代表著在查詢開始執行前，運算節點的數量就已確定，且無法隨著查詢負載的變化而調整。這種靜態特性在實際應用中顯現出其局限性。當查詢負載突然增加時，例如並發查詢數量激增或查詢本身的複雜度提升，預先配置的運算節點可能因資源不足而無法應對，進而導致查詢失敗或響應時間大幅延長。反之，當查詢負載較低時，叢集中過多的運算節點可能處於閒置狀態，造成資源利用率低下，無形中增加運營成本。這樣的問題在雲端環境中尤為突出，因為資源的高效利用與成本控制密切相關。

因此，針對 Presto 在雲端環境中靜態配置的不足，提出一個能夠透過 SQL 查詢語句來評估其查詢所需的資源以適應查詢負載的機制，此機制希望在 Oracle Cloud Infrastructure 的虛擬機實例 (VM Instance) 上建置 Presto 叢集，通過分析查詢的 SQL 語句與實際執行查詢的資源消耗，結合機器學習模型來預測查詢的資源需求，並基於此觸發 Auto Scaling 機制，調整運算節點的數量。同時，通過 Airflow 工作流程調度整合上述的工作，讓整個機制能夠自動化的進行，其能在高負載時提升查詢效能，還能在低負載時減少資源浪費，從而優化整體的資源利用率並降低雲端運營成本。

本研究旨在開發一個基於 SQL 查詢負載評估的動態調整機制，彌補現有方法在應對 Presto 在查詢時資源的不足的情況。通過此機制，我們期望在雲端環境中為 Presto 提供更高的查詢效率以及降低雲端運營成本，使其面對突發的高負載還是平穩的低負載，都能權衡查詢效能與成本效益。

### 1-3 問題定義與預期貢獻

Presto 作為一款高效能的大規模並行處理 (Massively Parallel Processing, MPP) 的 SQL 查詢引擎，廣泛應用於處理大規模數據集。然而，在資源受限且成本敏感的雲端環境中，Presto 叢集的運算節點數量，其配置採用靜態方的方式，代表節點數量在查詢任務執行前就已經預先設定，並在執行過程中保持不變。這種配置的方式，在面對動態變化的查詢負載時，會引發的資源利用效率低下與查詢效不穩定問題。此問題環境的挑戰會來自於兩個部分，其一是動態且不可預測的查詢負載，其中查詢請求的頻率、複雜度及數據處理量會隨時間顯著波動；其二是系統採用靜態配置的運算資源，尤其是 Presto 運算節點的數量和規格在查詢執行前已被固定。這個做法在靜態配置與動態負載之間的不相符，有著局限性。在高負載時，預設的運算資源可能不足以應對突然增加或是複雜語句的查詢需求，進而導致查詢佇列過長、響應時間延長，甚至查詢失敗。反之，在低負載時，大量已配置的運算節點則可能處於閒置狀態，這不僅造成運算資源的浪費，也增加了雲端的運營成本。

本研究結合 SQL 查詢語句的分析與機器學習資源預測模型，透過學習 SQL 語句特徵與歷史資源使用數據之間的關聯，預測查詢執行所需的資源。接著，根據預測結果動態評估所需的運算節點資源，並透過自動化流程系統完成從查詢接收、模型預測到資源調度的整體流程控制，實現 auto scaling 的機制，並預期藉由此機制解決 Presto 靜態配置的限制，相較於 Presto 官方的靜態配置方法，提出基於 SQL 查詢負載評估的 Auto Scaling 機制，使其能夠根據實際查詢負載調整運算節點數量，在高負載時提升查詢的效能，在低負載時減少資源浪費。此外，與現有基於閾值的 Auto Scaling 方法相比，本研究透過機器學習預測查詢資源需求，能夠在查詢執行前完成資源調整，避免調整滯後所帶來的資源瓶頸，從而提升查詢效率以及降低查詢執行時間，更預期能進一步提升 10-15% 的成本效益對其進行最佳化。

## 1-4 論文架構

1. 第一章 緒論：介紹研究背景、動機、目標、問題定義與預期貢獻。
2. 第二章 背景知識：闡述 MPP、Presto、Airflow、Terraform 等技術基礎。
3. 第三章 相關研究：分析 Presto、Auto Scaling、資源預測與 Workflow 管理的相關文獻，找出研究差距。
4. 第四章 系統設計：提出基於查詢分析與機器學習的 Auto Scaling 系統架構與設計細節。
5. 第五章 研究結果：比較靜態配置與 Auto Scaling 的查詢延遲、資源使用與雲端成本。
6. 第六章 結論與未來工作：總結研究貢獻並探討未來研究方向。
7. 第七章 參考文獻：列出研究相關參考文獻。

## 二、 背景知識

### 2-1 Massively Parallel Processing

Massively Parallel Processing (MPP) 興起於資料庫領域的應用研究，可追溯至 1980 年代末至 1990 年代初期 [17]。當時，研究者開始探討如何利用多處理器架構提升關聯式資料庫系統的效能，例如 Goodyear MPP 架構在資料庫管理系統中的應用，特別針對關聯式資料庫模型的平行處理潛力。該論文指出，MPP 架構能有效分散資料處理工作至多個節點，從而提升查詢效能，且關聯式模型因其數學基礎與結構化特性，特別適合平行處理。隨著資料規模呈指數級增長，傳統的 Symmetric Multiprocessing (SMP) 架構在面對大規模資料查詢與分析任務時，逐漸顯現延展性與效能瓶頸。MPP 架構因其優異的橫向擴展性與高併發處理能力，成為現代資料倉儲與分散式資料分析系統的核心基礎。

MPP 是一種允許系統中多個處理節點獨立且協同執行資料查詢任務的架構。每個節點擁有獨立的運算資源 (CPU、記憶體、儲存空間) 與資料切片，透過網路互連，並由協調者節點 (Coordinator) 負責查詢規劃、子任務分派與最終結果整合。查詢執行時，系統將原始查詢拆解為多個子查詢，分發至各節點進行並行處理，再由協調節點彙整為最終輸出。然而，當運算節點的記憶體不足以處理分配的子任務時，可能發生溢出 (spill) 至儲存空間的情況。此時，系統會將中間結果暫存至硬碟或固態硬碟，這將顯著增加 I/O 操作，導致查詢執行時間延長，影響整體效能。為緩解此問題，增加運算節點數量可有效分散工作負載，減少單一節點的記憶體壓力，從而降低對儲存空間的依賴，提升查詢效率。此外，某些查詢操作 (如簡單的篩選或聚合) 本身對記憶體需求較低，通常可在記憶體中完成，無需使用儲存空間。圖 1 為 MPP 的概念圖，從圖中我們可以看到資料分區與查詢任務分配的運作機制，旨在解決大規模資料處理中的效能瓶頸問題。此架構具備以下幾個特徵：

1. 資料分區：資料於寫入時即依據特定欄位進行水平分割，分散儲存在各節點之中，降低資料熱點與存取延遲。

2. 橫向擴展性：透過增加節點數量即可擴充系統整體處理能力與儲存容量，無須更動既有節點。
3. 容錯與高可用性：系統可透過與節點間重試機制實現容錯，確保部分節點失效不致於中斷整體查詢流程。
4. 查詢平行化：複雜查詢可分佈於多節點的資料片段，實現子任務同步執行與快速回應。

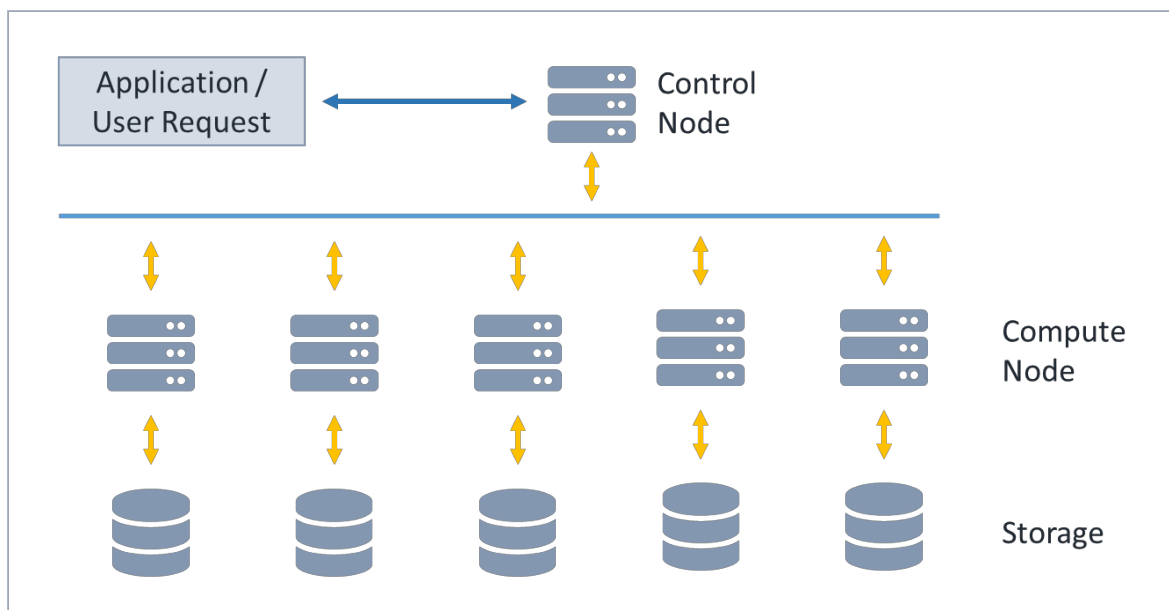


圖 1：MPP 概念圖

當前多數雲端資料倉儲與分散式查詢引擎皆基於 MPP 架構設計，例如 Amazon Redshift、Google BigQuery、Snowflake、Greenplum，以及開源系統如 Presto 與 Apache Druid 等。這些系統透過先進的查詢最佳化策略、資料壓縮技術與分散式執行引擎，有效支援 PB 級規模下的即時分析與交互式查詢。MPP 架構的採用顯著提升系統吞吐量與回應速度，推動資料密集型應用（如 OLAP（線上分析處理）、即時報表生成與機器學習模型訓練）在可接受時間內完成。例如，複雜的 JOIN 操作或大規模資料排序可能因記憶體需求過高而溢出至儲存空間，導致效能下降；但透過分配足夠的運算節點，可將任務拆分為較小的子任務，減少溢出風險。而簡單的 WHERE 條件篩選或 COUNT 聚合操作通常記憶體需求較低，可直接在記憶體中完成。因此，MPP 已成為處理現代資料密集應用的技術基礎之一。



儘管 MPP 提供高效並行處理，其資源分配多為靜態配置，難以動態應對查詢負載變化，可能導致效能瓶頸或資源浪費。此外，查詢的資源與查詢語法及資料特性密切相關。例如，涉及多表 JOIN 或高基數聚合的查詢可能因資料分布不均而難以預測資源需求，增加過度使用風險，突顯其在動態負載環境下的應用難度，也進一步強調 MPP 架構在解決大規模資料處理問題時的必要性與複雜性。

## 2-2 Presto

在大數據時代，傳統的資料倉儲系統往往無法因應異質性資料來源與多樣化查詢需求之挑戰。為解決此問題，Meta (Facebook) 於 2012 年開發了 Presto [3]，一種開源的分散式 SQL 查詢引擎，目的在提供一個高速、低延遲、異質數據源的即時查詢解決方案。圖 2 為 Presto 的架構圖，Presto 建構於 Massively Parallel Processing 架構之上，能在不中斷資料移轉或預先彙整的情況下，針對多個資料來源執行即席查詢（ad-hoc query），提升了資料查詢的靈活性與可擴展性。

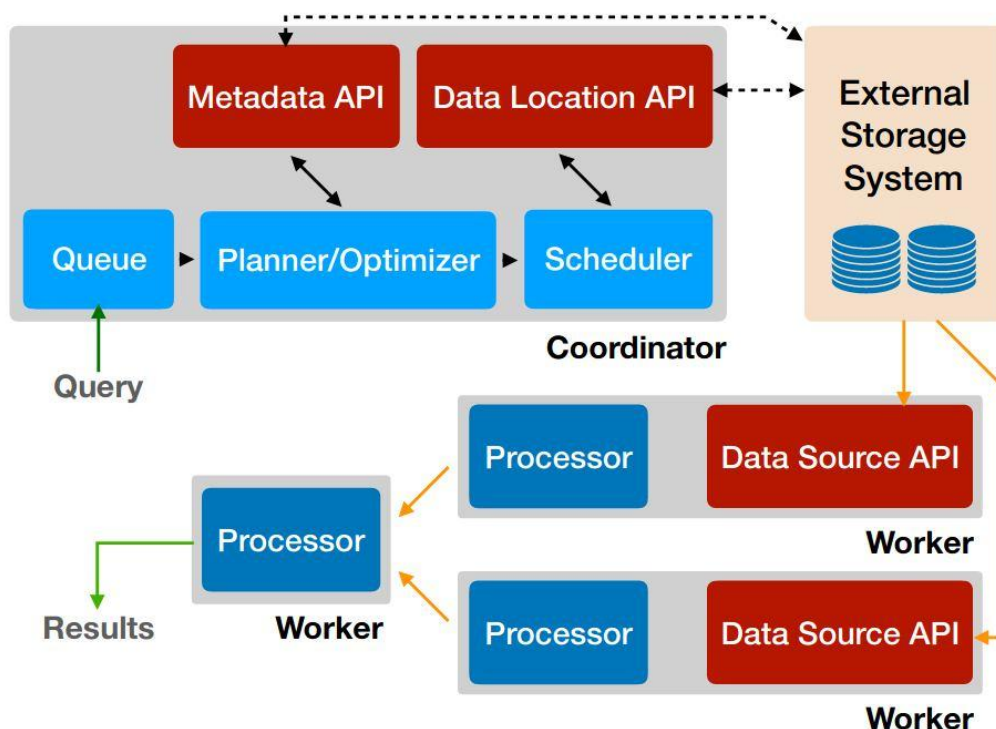


圖 2：Presto 架構圖

其系統架構採用分層設計，主要由三個元件構成：Coordinator 負責接收查詢請求並執行語法分析、查詢最佳化與階段劃分；Worker 則負責實際執行資料存取、過濾、聚合及 Worker 之間的資料交換操作，可依工作負載進行水平擴展；而 Connector Layer 提供對異質數據源（如：HDFS、MySQL、Kafka、MongoDB、S3...等）的存取能力，並將外部資料抽象為 Presto 可識別的結構，表 1 為 Presto 主要元件的說明。

表 1：Presto 主要元件

| 元件               | 功能描述                                                                           |
|------------------|--------------------------------------------------------------------------------|
| Resource Manager | 聚合來自所有 Coordinator 和 Worker 的資料並建立叢集全域視圖伺服器，其透過 Thrift API 與兩者進行通訊。            |
| Coordinator      | 負責接收查詢、進行語法分析與查詢最佳化，並將查詢拆解為多個階段，再分配給各 Worker 執行                                |
| Worker           | 執行實際運算任務，包括資料讀取、過濾、聚合與資料交換等操作。這些節點相互獨立，可橫向擴展以提升整體效能。                           |
| Connector        | 可將其理解為資料庫的驅動，允許 Presto 使用標準 API 與資源交互。                                         |
| Catalog          | 包含 Schemas 以及透過 Connector 引用數據源。                                               |
| Schema           | 是一種組織表的方法；Catalog 以及 Schema 定義了一組可以被查詢的表。                                      |
| Table            | 是一組無序的 Rows，其被組織成具有類型且被命名的欄位。                                                  |
| Statement        | 符合 ANSI 標準的 SQL 語句，其包含子句、表達式和述詞。                                               |
| Query            | 實例化以執行 Statement 的配置和元件，其包含 Stages、Tasks、Splits、Connector 以及協同工作已產生結果的其他元件數據源。 |
| Query Plan       | 是根據 SQL 查詢用於訪問和操作數據的一系列步驟。                                                     |
| Stage            | 為 Coordinator 用來建模分佈式查詢計劃的工具，其 root stage 負責聚合來自其他階段的輸出。                       |
| Task             | 分佈式查詢計劃被解構為一系列 Stages，然後轉換為 Tasks，然後 Tasks 對 Splits 進行操作或處理。                   |
| Split            | 為較大數據集的一部分，分佈式查詢計劃最低層的 Stage 通過 Connector 從 Split 檢索數據。                        |
| Driver           | 其對數據進行操作並組合運算符以產生輸出，然後該輸出由 Task 聚合並傳遞到另一個 Stage 的另一個 Task。                     |
| Operator         | 用於消耗、轉換和生成數據。                                                                  |
| Exchange         | 在查詢的不同階段在 Presto 節點之間傳輸數據。                                                     |

Presto 採用基於記憶體之執行引擎，避免將中間結果寫入磁碟，從而實現低延遲查詢。此外，其以向量化處理為基礎的引擎設計，亦提高了 CPU 使用效率與執行速度。在實際查詢過程，使用者透過 CLI、JDBC 或第三方商業智慧工具（如：Tableau 或 Superset）提交 SQL 查詢至 Coordinator 節點。Coordinator 隨即對查詢進行語法解析與語意分析，並完成邏輯與實體規劃，將整體查詢轉換為多階段的執行計畫，並將這些查詢階段會被分派至各個 Worker，以分散式方式進行資料擷取、過濾與處理；資料的存取則是透過 Connector 與底層資料來源互動。Worker 間可進行資料交換，以完成更複雜的聚合或 Join 操作。最後會將所有中間結果將被回傳至 Coordinator，並彙整為使用者所需的查詢輸出。

Presto 的叢集規模與各節點資源（如：CPU、記憶體）需在啟動前透過配置檔進行設置，並不具備依據即時查詢負載動態擴容或縮容的能力，若要擴增或縮減資源，通常必須手動新增或移除 Worker，並重新載入設定，才能讓新的資源配置生效。

## 2-3 Apache Airflow

面對工作流的排程與監控需求，傳統的腳本與排程工具（如：Cron）難以應對跨任務依賴關係、錯誤回復與工作視覺化等挑戰。為解決此問題，Airbnb 於 2014 年開發並開源了 Apache Airflow [4]，一套具備彈性與擴展性的工作流管理平台，用以定義、排程與監控資料導向任務。

Apache Airflow 採用模組化設計，整體系統如圖 3 Airflow 架構圖所示，由多個協同元件構成，表 2 為元件表，其核心設計理念為以程式碼定義工作流，使用者以 Python 腳本定義 DAG（有向無環圖），其中每個節點代表一個工作任務，邊則表示任務之間的依賴順序。此設計使工作流具備了可靈活定義任務依賴、條件邏輯與動態分支，並且 DAG 可作為模板重複使用，適用於批次處理、機器學習與報表流程等情境，此外，工作流邏輯可透過單元測試進行驗證，有助於流程上的整合。

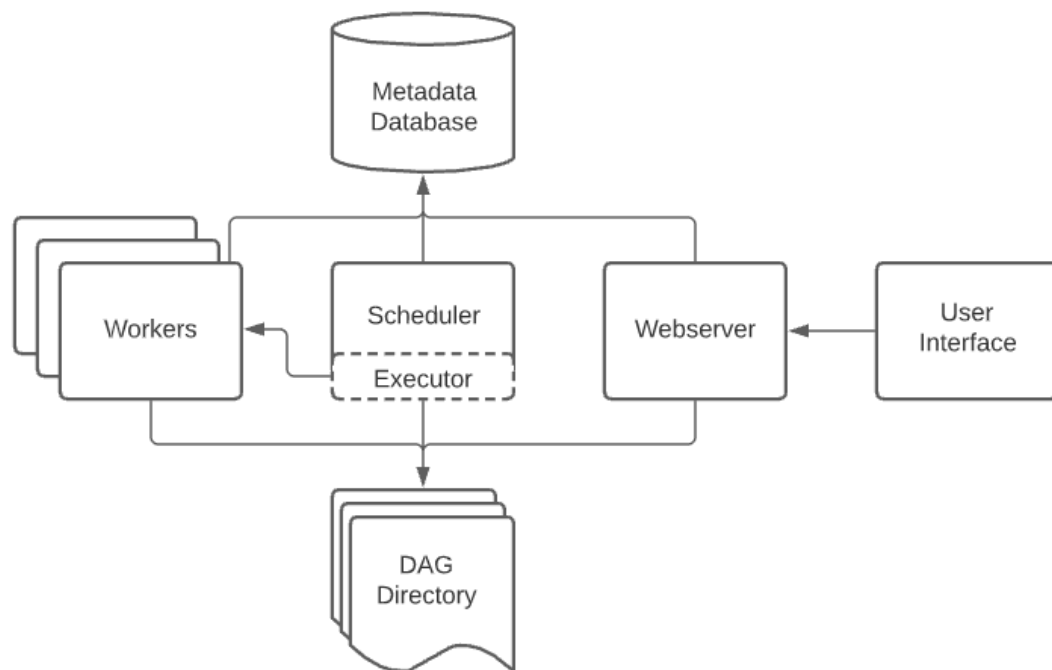


圖 3：Airflow 架構圖

表 2：Airflow 主要元件

| 元件                | 功能描述                                                                                           |
|-------------------|------------------------------------------------------------------------------------------------|
| Scheduler         | 負責週期性地掃描任務定義，評估其觸發條件與依賴關係，並將可執行的任務分發至 Executor。Executor 是調度器的配置屬性，而不是單獨的元件，並在 Scheduler 進程內運行。 |
| Webserver         | 提供方便的用戶界面，用於檢查、觸發和調試 DAG 和任務的行為。                                                               |
| DAG Directory     | 調度器會讀取此文件夾，以確定要運行的任務及其運行時間。                                                                    |
| Metadata Database | 儲存 DAG、任務狀態、排程記錄等關鍵資訊，常採用 PostgreSQL 或 MySQL 作為後端儲存。                                           |
| User Interface    | 提供用戶查看 DAG 及其任務的運行狀態、觸發 DAG 的執行、查看日誌，以及進行一些有限的調試和問題解決。                                         |

基於 Airflow 的 Operator、Hook 與 Sensor 機制，使用者得以將散落於不同系統與平台的元件串聯。如此一來，可以透過 DAG 整合查詢預測與資源調整，實現端到端的自動化工作流程。然而，現有 Airflow 應用多聚焦於任務調度，少見與資源預測或動態配置整合，限制了其能力。

## 2-4 Terraform

隨著雲端運算的普及與基礎設施動態化的需求提升，企業在建置與管理大規模分散式系統時，面臨資源配置複雜、環境不一致與部署流程重複等挑戰。為解決上述問題，「基礎設施即程式碼」（Infrastructure as Code, IaC）逐漸成為現代雲端開發的主流。Terraform 為 HashiCorp 所開發的開源工具[6]，是目前具代表性的 IaC 工具之一，支援多個雲端平台與自動化基礎設施部署，廣泛應用於雲端資源管理、自動化部署與基礎設施版本控制等領域。

圖 4 為 Terraform 的架構圖，其以宣告式語言來定義基礎設施，使用者透過其自有語言 HCL（HashiCorp Configuration Language）撰寫描述檔案，指定所需資源的最終狀態，可通過其動態調整 Oracle Cloud Infrastructure 虛擬機實例資源，支援 Presto 的 Auto Scaling，然而，其需與工作流系統整合，才能實現根據負載的資源調整。

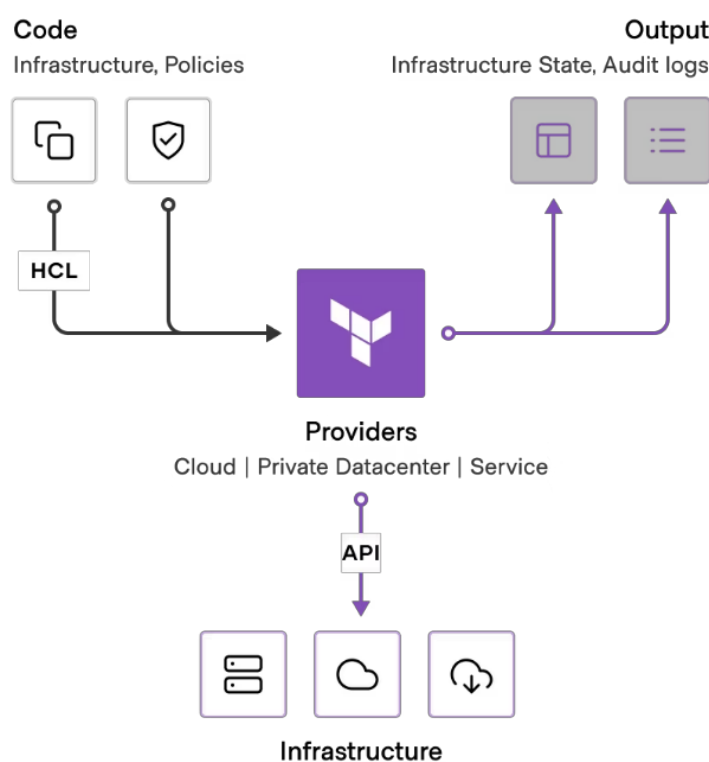


圖 4：Terraform 架構圖

表 3 來說明 Terraform 的元件；Terraform 會分析這些定義並生成一份執行計畫（Execution Plan），用以預覽即將變更的內容，確保部署過程可預期並且可控制。

表 3：Terraform 元件

| 元件           | 功能描述                                                                                                 |
|--------------|------------------------------------------------------------------------------------------------------|
| Providers    | Terraform 支援各種不同的 Infrastructure 供應商，如：AWS、Microsoft Azure、Google Cloud Platform、Oracle Cloud ... 等等 |
| Provisioners | 用於在資源創建後設置對應的狀態                                                                                      |
| Resources    | 代表 Infrastructure 中的一個單一元件，如：OCI VM Instance                                                         |
| Variables    | 模組的參數，如設置 OCI VM Instance 的類型和數量。                                                                    |
| Data Sources | 從現有的 Infra. 中檢索資料，如現有 OCI VM Instance 的列表                                                            |
| Outputs      | 顯示 Terraform 模組創建的 Infra. Component 的屬性，如：顯示 OCI VM Instance 的 IP 位置。                                |
| Modules      | 可重複使用的 Terraform 代碼區塊，用來創建類似的 Infra. Components。                                                     |
| State        | 紀錄目前 Infra. 的狀態，並在後續運行中與預期狀態進行比較，以確保 Infra. 一致。                                                      |
| Backend      | 儲存狀態的後端，其狀態包含有關當前 Infra. 狀態的資訊，如：哪些資源正在運行、哪些資源已經被創建、哪些資源被刪除等。                                        |

如圖 5 的 Terraform 工作流程圖所示，當使用者使用 Terraform 進行基礎設施自動化時，會先透過 HCL 撰寫 .tf 配置檔，定義所需資源與設定。接著執行 terraform init 初始化環境並下載 provider，然後用 terraform plan 預覽變更內容。確認無誤後，使用 terraform apply 建立或更新資源。

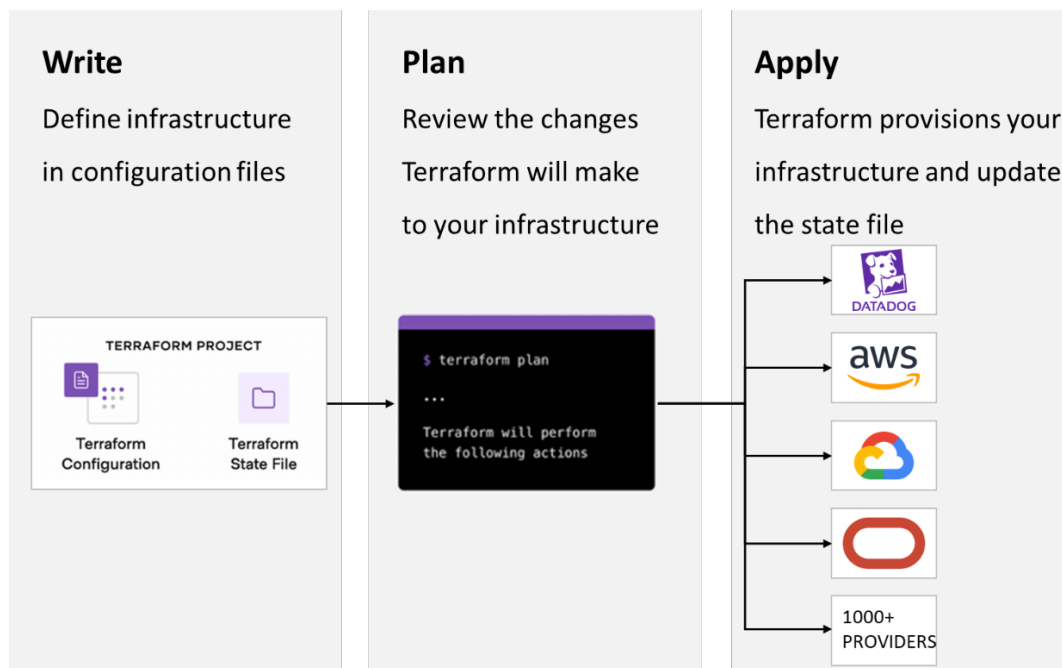


圖 5：Terraform 核心工作流程圖

## 三、 相關研究

### 3-1 Presto 分散式運算應用相關研究

在當今大數據驅動的商業與科技環境下，查詢系統的運行效率直接影響數據分析決策的即時性，這對查詢引擎在巨量數據處理方面的效能提出了更高挑戰。為因應此需求，Massively Parallel Processing (MPP) 架構被廣泛應用於查詢任務，透過將資料水平切分並分派至多個節點，以實現並行處理與計算負載的有效分攤。

Presto 作為一個由 Meta 發起並持續發展的開源分散式 SQL 查詢引擎 [7]，透過 MPP 架構展現出強大的資料處理能力，使其成為各大企業進行交互式分析的關鍵工具。根據 IBM 於 2023 年的報告，Meta 所部署的 Presto 叢集每天執行超過 30,000 次查詢，處理規模達 PB 等級，顯示其在企業級應用中的重要地位。

Santhosh Gourishetti [8] 的研究中提出了一套針對 Presto 的工作負載優化框架，涵蓋記憶體管理、平行處理策略、儲存連接器設定、JVM 調校以及資源池設計，旨在縮小預設的靜態配置與實際部署之間的落差，並提供在不同部署情境下的效能優化建議。然而，Presto 採用靜態配置，其 Worker 數量與 CPU、記憶體資源均需事先設定，無法根據實際負載動態調整。因此，為因應資源需求的短期波動，透過導入 Auto Scaling 機制，不僅降低查詢的延遲且能提升系統的穩定度，從而進一步降低成本。

### 3-2 Auto Scaling 應用相關研究

Auto Scaling 是雲端平台中關鍵的資源調控機制，其核心目標為在負載高峰時自動擴展計算節點，在低峰時自動釋放閒置資源，以達成效能與成本之間的平衡，而雲端供應商普遍採用 CPU 利用率、記憶體占用率等基礎硬體指標作為觸發條件。

Saleha Alharthi 和 Afra Alshamsi et al. [10] 在其研究中指出，現行的自動擴展技術雖然成熟，但多半忽略了應用層邏輯特性，無法根據特定應用行為進行主動式預測調整。此外，作者指出現有 Auto Scaling 策略難以因應高變動性、短時爆發的工作負載，其容易導致服務品質下降或資源浪費。然而，在資料查詢領域，查詢引擎本身具

有高度語意結構，其行為在查詢前即能部分預知，但這類語意資訊尚未被整合進資源調配流程中；如 SQL 查詢這類執行時間短但資源瞬間拉高的任務。若能預先評估查詢的資源需求，進而主動觸發資源調整，將有助於提升擴展策略的即時性與準確性。

然而，現有的 Auto Scaling 多半建立對於當前系統資源需求的即時監控與短期預測，或者僅於資源不足的情況發生後，才會被被動地進行擴展，此類方法雖具備一定的彈性，能因應部分負載的變化，但在面對突發或是不可測的流量時，反應速度時常不足，這使得現行方法對於未來負載的預判能力有限；因此，若 Auto Scaling 的觸發機制能結合 SQL 查詢語句的分析，在執行查詢之前預先知道所需的運算資源，再進一步做資源上的調整，得以應對大規模數據處理的彈性需求。

### 3-3 查詢語法分析與資源需求預測相關研究

查詢優化器在分散式資料處理系統中扮演關鍵角色，其核心功能是預估不同執行計畫的成本並選擇最有效率者執行。傳統上，成本預估方法多半建立在資料統計資訊與操作類型（如 Join、Filter）的結構性分析。然而，這類方法雖能協助選擇較佳的執行路徑，卻難以準確反映查詢執行時動態所需的實際運算資源（如 CPU、Memory）。

Chunxu Tang 和 Beinan Wang et al. [11] 在其研究中提出一套 SQL 查詢成本的預測模型，透過分析 SQL 查詢中的語法結構、過往執行紀錄與操作型態，建立查詢成本的預測機制。此研究以機器學習模型對查詢的 AST（Abstract Syntax Tree）與執行計畫特徵進行向量化處理，再結合歷史查詢表現進行預測，其繞過傳統數據庫需要依賴查詢計畫的計算瓶頸，實現 97% 以上的預測準確率。

然而，作者雖然在 CPU Time 及 Memory 的預測上取得了顯著成果，但其主要目標在於提升成本估算的準確性，並未探討如何將此預測結果應用於動態的資源擴展（Auto Scaling）觸發機制。因此，在實際的 Presto 查詢場景中，我們需要考慮如何將查詢資源需求的預測模型與雲端平台的 Auto Scaling 機制相結合。本研究會專注在如何透過 Workflow management system 建置一套完整的工作流程，能在查詢提交後、執行開始前，根據 SQL 查詢語句的資源評估，觸發 Presto 的資源調整。



## 四、系統設計

### 4-1 系統架構

在當今大數據時代，隨著數據量的快速增長，高效能的數據查詢工具顯得重要。Presto 作為一款開源的分佈式 SQL 查詢引擎，憑藉其高效能與靈活性，已廣泛應用於企業。然而，在計算資源有限的環境當中，資源會被多個應用程序共同使用，不能讓 Presto 占滿所有資源，且 Presto 無法在查詢期間自動增加或減少運算資源，導致當查詢負載增加時，可能因運算資源不足導致查詢延遲高甚至失敗；相反地，當查詢負載降低時，閒置的資源則造成浪費，進而讓系統的運營成本提升。

為解決上述問題，本研究提出了一種基於 SQL 查詢語句進行資源需求評估的 Presto Auto Scaling 機制，旨在根據 SQL 語句的複雜度，在查詢執行前自動調整 Presto 的運算資源，以最佳化查詢效能並降低運營成本。

要實現上述的機制，我們需要思考如何根據 SQL 查詢語句的複雜度評估其資源需求、如何基於預測的資源開銷與歷史查詢數據，估算需要的 Presto 運算資源、如何以自動且一致的方式配置 Presto 運算資源、如何透過設計並且控管 Workflow，以及讓用戶如何觸發 Workflow。

綜合考量以上的問題，我們以 presto-query-predictor module 為基底建立 SQL 語法資源開銷的預測模型，並藉由 REST API 的呼叫來調用模型，再根據資源的開銷預測以及過去歷史查詢結果的分析，來估算 Presto 需要的運算資源，並調整 Presto 運算資源使其與估算的資源量一致，再透過 Airflow 將上述的流程整合進 DAG 中，以利整個機制的控管，使用者則經由 HTTP 的方式，發送帶有 SQL 語句的請求，觸發 DAG 執行工作流程。

圖 6 為系統架構圖，其中灰色部分為本研究需開發的元件，其於為現有系統運行的必要元件 Airflow 負責協調工作流程，依序調用 Query Prediction Service、Resource

Estimation Service 與 Infrastructure Provisioning Service，實現 Presto Auto Scaling，表 4 為系統元件的簡述，詳細的內容會在後續的章節進行說明。

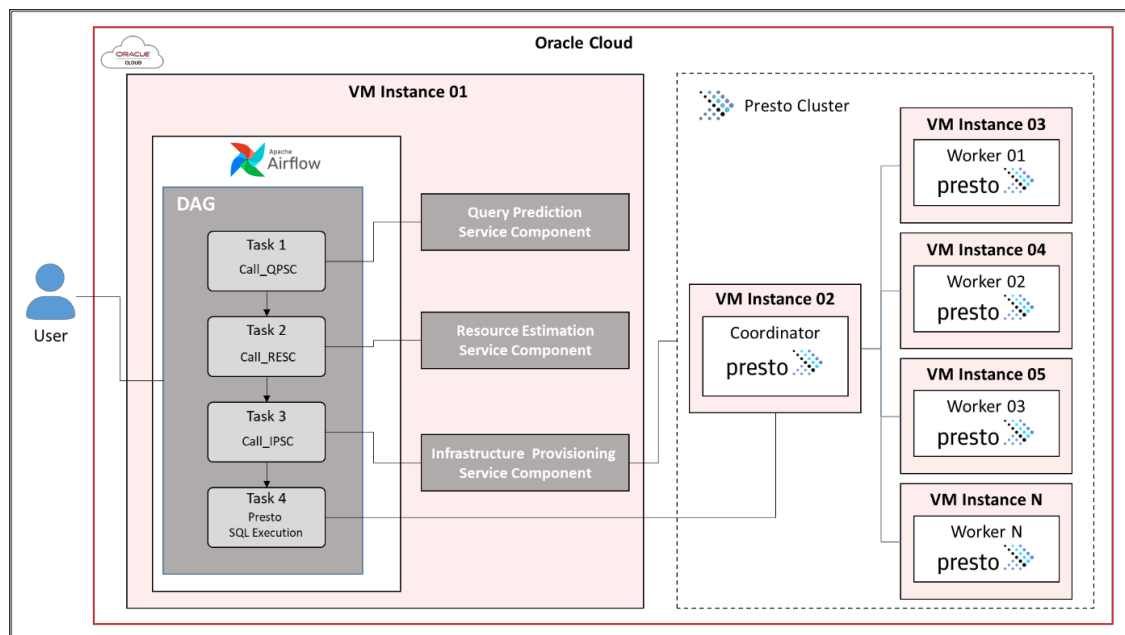


圖 6：系統架構圖

表 4：系統元件

| 元件名稱                                | 功能描述                                                                                                                                                                                          |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VM Instance                         | 雲端虛擬化計算環境，用於部署與運行系統各元件。包括現有 元件：VM Instance 01 運行 Airflow，VM Instance 02 運行 Presto Coordinator ... 等。                                                                                          |
| Apache Airflow                      | 運行於 VM Instance 的工作流程編排工具，通過 DAG 管理系統流程。DAG 包含四個任務：調用 Query Prediction Service、Resource Estimation Service、Infrastructure Provisioning Service 及執行 Presto SQL 查詢，確保流程自動化與順序執行，並由用戶 HTTP 請求觸發。 |
| Query Prediction Service            | 基於 presto-query-predictor 模組預測 SQL 語句所需的資源量，並通過 REST API 提供預測結果。使用 DAG 整合於 Task 1。                                                                                                            |
| Resource Estimation Service         | 根據 Query Prediction Service 的預測結果，估算所需的 Presto 運算資源，並透過 Airflow DAG 整合於 Task 2。                                                                                                               |
| Infrastructure Provisioning Service | 負責配置 Presto 運算資源。根據估算結果，調整 Presto Worker 數量，以滿足查詢需求。並透過 Airflow DAG 整合於 Task 3。                                                                                                               |
| Presto Cluster                      | 由 1 個 Coordinator 以及多個 Worker 組成，其協同工作來解析、規劃、排程及平行化執行查詢任務。                                                                                                                                    |

## 4-2 Resource Estimation Service Component

Resource Estimation Service Component (以下簡稱 RESC) 是本研究中 Presto Auto Scaling 關鍵元件，主要功能是將預測模型的輸出轉換為實際的資源分配策略。RESC 本身不進行預測，而是在預測結果產出後進行資源需求的估算。它接收來自 Query Prediction Service Component 預測模型的兩個主要指標：CPU Time 與 Memory，並計算出所需的 Presto Worker 數量，由於 CPU Time 和 Memory 的資源需求依賴不同因素（CPU 使用與任務並行度高度相關，而記憶體需求與數據量和查詢操作類型有關），故 RESC 採用差異化的估算策略，以利降低資源不足的風險。

RESC 的設計有四個部分，首先將資源需求離散化，將連續型預測結果 CPU Time 與 Memory 對應至統計區間中；接著採用區間式估算，以歷史數據分析的區間代表值計算查詢所需的 CPU Time 與 Memory，進而決定所需的 Worker 數量；可依據個別查詢的需求自動調整 worker 數，並設定最小值以維持系統穩定；最後，在 Memory 估算中納入資源安全係數，以因應 Memory 使用分布不均或突發的負載變化，提升整體系統的穩定。

### 4-2-1 CPU 核心數資源估算

並行度 (Parallelism) 其值為  $totalCpuTime / executionTime$ ，反映查詢執行期間的平均併發數量。根據 Presto 官方文件 (Presto Concepts)，並行度表示查詢過程的並行執行特性，因而是估算 CPU 核心數的理想指標。與傳統方法不同，本研究不依賴 CPU Time 的代表值進行換算，而是採用並行度作為 CPU 核心需求。透過歷史查詢數據的統計分析，如表 5 所示，RESC 將並行度依照 CPU Time 區間分段，並選取修剪平均值作為每個區間的代表值。採用並行度的依據為：首先，並行度反映了工作負載在 CPU 資源上的分佈，無需額外轉換；其次，歷史數據的實證分析顯示，修剪平均值並行度能有效捕捉典型查詢行為。

表 5：CPU Time 區間對應 Parallelism

| CPU Time (s)  | Parallelism |
|---------------|-------------|
| [ 0, 30 ]     | 2           |
| [ 30, 300 ]   | 17          |
| [ 300, 1800 ] | 42          |
| [ 1800, ]     | 31          |

#### 4-2-2 記憶體資源估算

與 CPU 核心數估算不同，Memory 需求與並行度無直接關聯，而是取決於查詢處理的數據量及操作類型（如：連接、聚合）。因此，RESC 會針對 Memory 採用區間估算方法。根據歷史數據的統計分析，如表 6 所示，我們將其分為離散的 Memory 區間，每個區間選取其代表值，以應對 Memory 的需求。為避免無預警的 Memory 使用量，RESC 應用 1.5 倍的安全係數，以確保因應 Memory 的突發變動。

表 6：Memory 區間對應代表值

| Peak Memory (MB) | 代表值 (MB) |
|------------------|----------|
| [ 0, 100 ]       | 50       |
| [ 100, 500 ]     | 300      |
| [ 500, 1000 ]    | 750      |
| [ 1000, ]        | 1000     |

相較於基於閾值的 Auto Scaling [9]，RESC 利用查詢特徵預測，實施資源分配。本研究 Worker 數量計算公式如下：

選定的並行度代表值直接作為所需的 CPU 核心數

$$RequiredCPUCores = ParallelismRepresentativeValue$$

考量每個 Worker 提供 8 個 CPU 核心，根據 CPU 需求推估所需 worker 數量。

$$N_{CPU} = \frac{Require\ CPU\ Cores}{8}$$

而每個 Worker 節點提供 20 GB (20480 MB) 記憶體，並加入 1.5 倍安全係數，計算所需 Worker 數量。

$$N_{Memory} = \frac{Memory \times 1.5}{20480}$$

綜合 CPU 與 Memory 資源的估算，取其最大值為最終所需之 Worker 數量

$$N = \max(N_{CPU}, N_{Memory})$$

## 4-3 Query Prediction Service Component

Query Prediction Service Component (QPSC) 基於 presto-query-predictor 預測 SQL 查詢的 CPU Time 與 Memory 需求。相較於 Twitter 對 SQL 查詢成本預測的研究 [12]，本研究整合動態資源調整，提供端到端解決方案。本子章節會描述訓練資料集的組成、特徵工程方法、資料表大小與操作類型的影響，以及訓練集與測試集的相似性分析，以說明模型的預測機制與性能。

### 4-3-1 Data Pre-Processing

在 QPSC 中，預測模型的原始訓練資料我們使用 python 腳本來執行查詢，並且取得查詢結果的數據。透過調用 stats 屬性，我們能夠獲取查詢執行後的統計資訊，包括查詢 ID、查詢語法、執行時間、CPU Time 等關鍵指標。然而，這些數據在原始形式下並不適合直接用於模型訓練，因為其中許多本應為數值的欄位（如：CPU 使用時間、記憶體使用量等）是以字串形式表示的，例如 “totalCpuTime”: “12s”。因此需要將這些字串形式的數值欄位轉換為相應的數值類型，以利後續的資料分析和模型訓練的進行。以下是資料預處理流程，其預處理完的資料格式會在表 7 進行說明。

#### 1. 資料提取

- 透過 python 腳本執行查詢並透過 stats 獲取查詢結果的數據。這些數據以字典形式返回，包含多個與查詢執行相關的欄位，如查詢 ID、查詢語法、執行時間、CPU Time、記憶體使用量等。

## 2. 欄位識別與轉換

- 由於部分欄位在原始數據中以字串形式呈現，並且通常包含單位（如 "12s" 表示 12 秒），我們需要對這些欄位進行數據類型轉換。
- 首先，識別出需要轉換的欄位，這些欄位在本研究中包括但不限於：
  - i. totalCpuTime：CPU 使用時間，原始形式如 "12s"
  - ii. peakTotalMemoryReservation：峰值記憶體使用量，原始形式如 "1.2GB"
- 對於每個需要轉換的欄位，執行以下步驟：
  - i. 提取數值部分：例如，從 "12s" 中提取 12。
  - ii. 識別並處理單位：例如，"s" 表示秒，"GB" 表示 Gigabytes 等。
  - iii. 將數值轉換為統一的數值類型：例如，將 CPU Time 轉換為秒，將記憶體使用量轉換為 MB。

## 3. 資料標準化

- 為確保不同查詢數據之間的一致性，對於欄位進行標準化處理。例如，將所有記憶體使用量統一轉換為 MB 單位。

## 4. 資料儲存

- 將經過預處理的數據以 CSV 格式儲存，作為後續資料分析和模型訓練的基礎。

經過資料的預處理，其資料集的組成包含 7,000+ 筆 SQL 查詢，來源為模擬企業級資料庫環境中的真實查詢記錄，涵蓋人力資源（HR）、財務（Finance）及銷售（Sales）等業務場景。查詢由 Presto 執行於固定規模的資料表生成，透過 Python 腳本收集執行結果。資料集涵蓋以下查詢類型：

- 簡單查詢（15%）：如單表 SELECT 或簡單 WHERE 過濾，例：  
"SELECT employee\_id, first\_name FROM employees WHERE department\_id =

10;" 執行結果：CPU Time 約 12.5 秒，記憶體使用量約 50 MB，執行時間約 15 秒，輸出數據大小約 2 MB，輸出行數約 50 行。

- 中等複雜查詢（60%）：包含單表或雙表 JOIN，結合 WHERE、GROUP BY 或 ORDER BY，例：" SELECT d.department\_name, COUNT(e.employee\_id) AS emp\_count FROM employees e JOIN departments d ON e.department\_id = d.department\_id WHERE e.hire\_date > '2020-01-01' GROUP BY d.department\_name;" 執行結果：CPU Time 約 450.2 秒，記憶體使用約 750 MB，執行時間約 480.5 秒，輸出數據大小約 15 MB，輸出行數約 120 行。
- 高複雜度查詢（25%）：包含多表 JOIN、子查詢或 CTE，例：" WITH dept\_avg AS ( SELECT department\_id, AVG(salary) AS avg\_salary FROM employees GROUP BY department\_id ) SELECT d.department\_name, da.avg\_salary FROM departments d JOIN dept\_avg da ON d.department\_id = da.department\_id WHERE da.avg\_salary > 5000 ORDER BY da.avg\_salary DESC;" 執行結果：CPU Time 約 2,100 秒，記憶體使用量約 1.5 GB，執行時間約 2,300 秒，輸出數據大小約 10 MB，輸出行數約 80 行。

表 7：資料預處理後的資料欄位示例

| 字段名稱                       | 數據類型 | 描述                     |
|----------------------------|------|------------------------|
| catalog                    | 字串   | 查詢使用的目錄名稱。             |
| executionTime              | 浮點數  | 查詢實際執行所花費的時間。          |
| peakTotalMemoryReservation | 浮點數  | 查詢達到的峰值總記憶體預留量。        |
| query                      | 字串   | 執行的 SQL 查詢語句。          |
| queryId                    | 字串   | 查詢的唯一標識符。              |
| queryType                  | 字串   | 查詢的類型。                 |
| schema                     | 字串   | 查詢使用的 schema 名稱。       |
| shuffledDataSize           | 浮點數  | 查詢執行過程中數據在節點間交互的總數據大小。 |
| source                     | 字串   | 提交查詢的客戶端或來源應用程式。       |
| totalCpuTime               | 浮點數  | 查詢執行過程中消耗的總 CPU 時間。    |
| user                       | 字串   | 執行查詢的用戶名稱。             |

### 4-3-2 Data Explorer

在進入到模型訓練階段前，我們通過 EDA 來分析這 7,000 + 筆預處理完的查詢結果數據，為後續的模型訓練提供基礎。

#### 4-3-2-1 資料分析

為有效分析和預測 SQL 查詢的資源消耗，本研究對查詢執行記錄中的 totalCpuTime（總 CPU 時間，單位：秒）和 peakTotalMemoryReservation（峰值記憶體保留，單位：MB）進行離散化處理，將連續數據轉換為離散區間，以簡化查詢性能分類，為特徵工程和模型訓練提供依據。本方法參考 Twitter 的 SQL 查詢成本預測研究 [12]，結合統計分佈來定義 CPU Time 以及 Memory 對應的區間範圍。

在資源需求的區間劃分上，首先確認最大邊界值以涵蓋所有查詢的資源使用範圍。接著觀察數據分佈後，發現 totalCpuTime 的最大值為 6,084.0 秒，代表極端 CPU 消耗的情況；而峰值記憶體保留量（peakTotalMemoryReservation）的最大值則為 10,874.88 MB，其為極端記憶體使用量。這些最大值被設定為區間的上限，確保模型在預測時能覆蓋所有可能的查詢負載。

對於 CPU Time 的區間劃分，結合數據分佈特徵與參考文獻 [12] 中的經驗閾值進行設計。數據顯示，約 70% 的查詢 CPU Time 低於 300 秒，而在長尾部分則延伸至 6,084.0 秒。基於此，將 CPU Time 劃分為四個層級區間：[0, 30 秒) 為輕量級查詢，通常是簡單的 SQL；[30, 300 秒) 則是中等 CPU 消耗，接近數據的 25% 分位數，涵蓋中等複雜度的查詢；[300, 1800 秒) 屬於高 CPU 消耗，對應 50% 至 75% 分位數的範圍，為資源密集型的操作；[1800 秒,  $\infty$ ) 則針對長尾分佈中的極端情況。

在記憶體使用的劃分上，數據顯示其分佈相對均勻。參考四分位數統計結果與文獻 [12] 提出的 1 MB 低記憶體閾值，將峰值記憶體保留量分為四個區間：[0, 100 MB) 表示低記憶體使用，接近 25% 分位數；[100, 500 MB) 為中等記憶體使用，涵蓋約 50% 分位數的查詢；[500, 1000 MB) 代表高記憶體需求，接近 75% 分位數；而 [1000 MB,  $\infty$ ) 則對應極端記憶體使用的情況。此劃分策略充分考慮了記憶體使用的分佈特徵，確保在模型分類時具備良好的實用性與判別能力。



基於上述區間，本研究對 5,327 筆查詢執行記錄的子集進行統計分析，涵蓋 totalCpuTime 和 peakTotalMemoryReservation 的分佈特性。統計摘要顯示，totalCpuTime 範圍從 0.15 秒到 6,084.0 秒，平均值為 649.28 秒，標準差為 813.97 秒，四分位數為 285.0 秒（25%）、455.4 秒（50%）、759.6 秒（75%）；peakTotalMemoryReservation 範圍從 0.0 MB 到 10,874.88 MB，平均值為 878.54 MB，標準差為 1,374.22 MB，四分位數為 89.585 MB（25%）、493.59 MB（50%）、988.77 MB（75%）。這些指標顯示高度變異性，特別是在高資源消耗的查詢中。

- 資源使用分佈：

- CPU Time 分佈：

- ◆ 圖 7 展示 CPU Time 的直方圖，涵蓋 7,000+ 筆查詢數據，分為四個區間：[0-30s]、[30s-300s]、[300s-1800s]、[1800s+]。數據分佈如下：

- [0-30s]：235 筆（2.99%）
      - [30s-300s]：2381 筆（30.28%）
      - [300s-1800s]：4932 筆（62.72%）
      - [1800s+]：315 筆（4.01%）

- ◆ 直方圖顯示大多數查詢（62.72%）的 CPU Time 集中在 [300s-1800s] 區間，少數查詢（4.01%）的 CPU Time 超過 1800 秒。

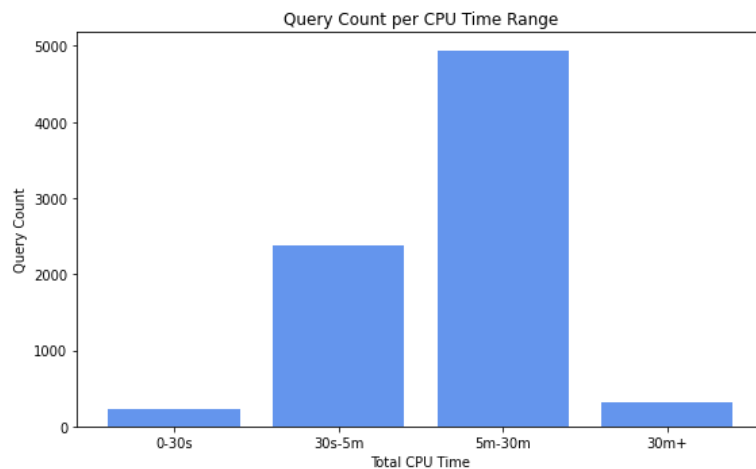


圖 7：CPU Time 分佈直方圖

Memory 使用量分佈：

◆ 圖 8 展示 Memory 使用量的直方圖，分為四個區間：[0-100MB]、[100-500MB]、[500MB-1GB]、[1GB+]。數據分佈如下：

- [0-100MB]：1886 筆 (23.99%)
- [100-500MB]：2777 筆 (35.32%)
- [500MB-1GB]：1750 筆 (22.26%)
- [1GB+]：1450 筆 (18.44%)

◆ 直方圖顯示 Memory 使用量分佈較為均勻，[100-500MB] 區間的查詢最多 (35.32%)，而約 18.44% 的查詢 Memory 需求超過 1GB。

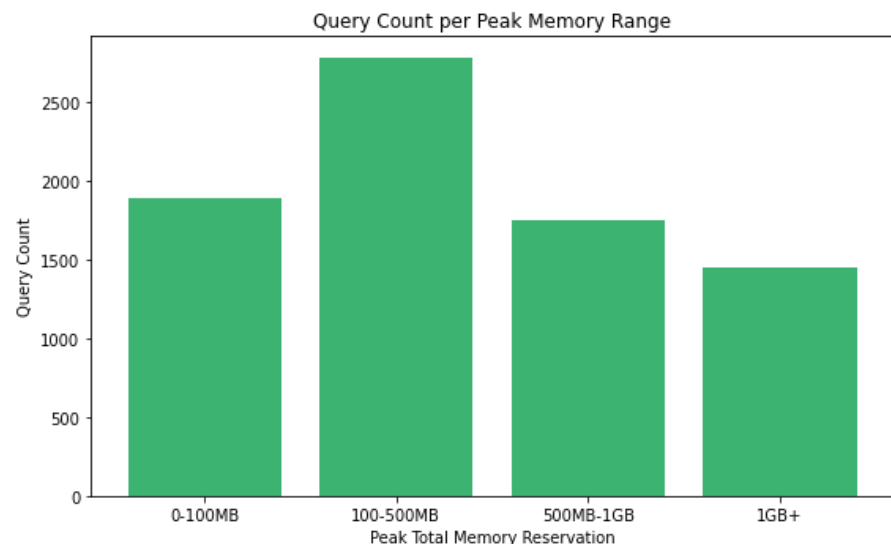


圖 8：Memory 分佈直方圖

● 資源使用關係

■ 圖 9 為一個散點圖，呈現 CPU Time 與 Memory 之間的關係。該圖包含 5327 筆查詢數據，大部分查詢的 CPU Time 小於 500 秒，Memory 小於 1000 MB，集中在圖表的左下角。然而，少數查詢表現出極高的資源需求，例如 CPU Time 達到 6084 秒，Memory 達 10.8 GB，這些數據點位於圖表的右上角。

- 散點圖顯示 CPU Time 與 Memory 之間無明顯線性相關性，但隨著 CPU Time 增加，Memory 使用量的變異性顯著增大。表示資源使用受查詢複雜度的影響較大，而非簡單的線性關係。

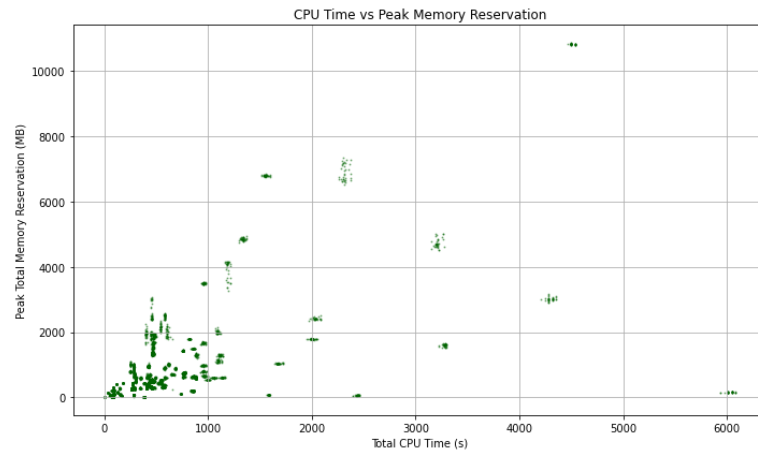


圖 9：CPU Time 與 Memory 的散點圖

- SQL 語法特徵分析：

- 語法特徵數量統計：

- ◆ 圖 10 展示每筆查詢的語法特徵數量分佈，涵蓋 5327 筆查詢數據，其語法特徵數量被分為 8 個 SQL 關鍵字進行統計。
    - ◆ 根據直方圖統計的結果，SQL 語法關鍵字存在多種模式。大多數查詢的特徵具有 WHERE、GROUP BY、ORDER BY，顯示多數的查詢帶有條件式排序以及聚合的功能。少數查詢則使用聯集以及 CTE
    - ◆ 特徵數量的分佈與資源使用可能存在關聯。結合圖 7 和 圖 8 的資源分佈，如 SQL 語句具有特定的特徵，其查詢可能與高 CPU Time 或 Memory 需求相關，會在之後的實驗結果進行分析以及比對。

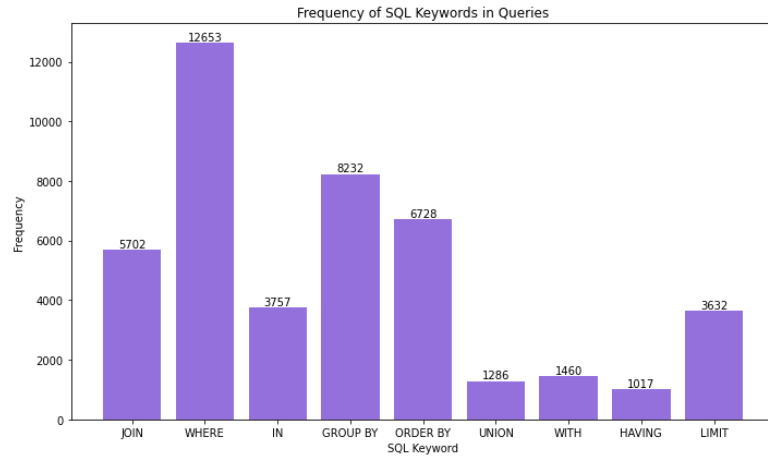


圖 10：SQL 語法特徵數量分佈直方圖

#### 4-3-2-2 特徵工程

為將 SQL 查詢語法轉換為機器學習模型可處理的特徵向量，本研究採用 TF-IDF 方法進行特徵工程，提取與資源需求高度相關的語法特徵。

首先對查詢語法進行標準化處理，將其轉為小寫並分解為詞彙單元，涵蓋關鍵字（如 SELECT、JOIN、WHERE、GROUP BY、ORDER BY）、運算符（如 =、>）以及特定函數（如 COUNT、SUM、AVG、RANK）。同時，提取與資源需求相關的結構特徵，包括 JOIN 的數量（從單表到多表）、子查詢層次、CTE 的使用情況以及聚合函數的數量。例如，JOIN 操作涉及多表資料匹配，通常會增加 CPU Time 與記憶體需求；GROUP BY 與聚合函數需要進行分組計算。

再透過 TF-IDF 向量化計算每個詞的權重，其中詞頻（TF）反映詞彙在單一查詢中的出現頻率，而逆文檔頻率（IDF）則降低常見詞彙（如 SELECT）的權重，並突顯對資源需求影響較大的詞彙（如 JOIN、GROUP BY、CTE）。例如，多個 JOIN 關鍵字在向量化後的 TF-IDF 分數往往較高，反映其對資源需求的顯著影響；而 SELECT 由於幾乎在所有查詢中出現，其權重則相對較低。最終生成的多維度的特徵向量，能同時涵蓋關鍵字頻率、函數使用情況以及結構複雜度等資訊。

進一步的關聯性分析顯示，JOIN 的數量與記憶體需求呈現正相關，多表 JOIN 查詢往往增加資料匹配成本，例如三張表進行 JOIN 查詢的記憶體需求常超過 1GB（參

考圖 8)。聚合函數（如 COUNT、AVG）在多欄位 GROUP BY 條件下，會顯著增加記憶體使用量。子查詢與 CTE 則增加語法解析與執行的複雜度，造成 CPU Time 的明顯上升，其中部分 CTE 查詢的執行時間甚至超過 1800 秒（參考圖 7）。相關性分析結果（圖 9）亦表明，JOIN 數量與記憶體需求的變異性正相關，而子查詢與 CTE 對 CPU Time 的影響更為顯著。

#### 4-3-2-3 操作類型分析

SQL 查詢的操作類型（如 SELECT、JOIN、GROUP BY）對 CPU Time 和記憶體需求有顯著影響。分析顯示（圖 9），JOIN 操作數量的增加導致數據匹配成本上升，放大 CPU Time 和記憶體使用量；GROUP BY 和聚合函數（如 COUNT、AVG）因分組計算而增加記憶體需求；子查詢和 CTE 提升語法複雜度，顯著增加 CPU Time。此分析為特徵工程提供依據，確保模型能捕捉操作類型的資源需求特性。

#### 4-3-3 Model Training

在完成資料預處理與特徵工程之後，本研究進一步進行模型訓練，使用 presto-query-predictor 作為核心訓練模組，預測 SQL 查詢的資源使用情況。presto-query-predictor 是一個 Presto 生態系的 Python 模組，其目標在通過機器學習技術預測 Presto 查詢的資源需求。該模組提供一個 Machine Learning Pipeline，涵蓋從資料加載到模型評估的六個階段，並提供一個查詢預測的 Web 服務，用於預測查詢的 CPU Time 和 Memory 的使用情況，以支援系統的資源調度和最佳化。其中，presto-query-predictor 的 Machine Learning Pipeline 包含以下六個階段：

1. 資料加載：將 4-3-1 預處理所產生的 CSV 檔
2. 資料轉置：在加載原始資料後，對資料集進一步處理，將 SQL 語法轉換為小寫以確保一致性，並根據查詢的實際資源使用情況創建預測標籤，將 CPU Time 和 Memory 分為不同的類別區間。
3. 資料拆分：將轉置後的資料分為訓練集和測試集，以利模型訓練和評估，其採用標準分割比例 80% 訓練集，20% 測試集來確保模型的泛化能力。

4. 向量化：模組提供兩種向量化方法，將 SQL 查詢語法轉換為機器學習模型可處理的特徵向量
  - i. DataCountVectorizer：基於標記計數的方法，計算每個詞在查詢語法中的出現次數，將文本轉換為特徵向量，適用於簡單的文字分類任務。
  - ii. DataTfidfVectorizer：基於 TF-IDF（詞頻-逆文檔頻率）的方法，考慮詞語在所有查詢中的相對重要性，將文字轉換為特徵向量，適用於需要突出關鍵詞的場景。
5. 模型訓練：模組支援三種分類演算法，用於構建資源預測模型，並透過 Tuning 找出訓練的超參數，將其回饋到模型訓練參數當中，如圖 11 所示。
  - i. RandomForestClassifier：基於 scikit-learn 的隨機森林分類器，通過集成多棵決策樹提升預測準確性，特別適合處理高維數據。
  - ii. LogisticRegressionClassifier：基於 scikit-learn 的邏輯回歸分類器，適用於二元分類問題，具有簡單且高效的特點。
  - iii. XGBoostClassifier：基於 xgboost 的梯度提升分類器，以高效性和優異性能著稱，特別適合大規模數據集和複雜預測任務。

```
label: cpu_time_label
feature: query
vectorizer:
  type: tfidf
  params:
    max_features: 100
    min_df: 1
    max_df: 0.8
  persist: true
  persist_path: models/vec-cpu.bin
test_size: 0.2
classifier:
  type: XGBoost
  params:
    max_depth: 2
    objective: 'binary:logistic'
    persist: true
    persist_path: models/model-cpu.bin
```

圖 11：模型訓練的參數

6. 模型評估：使用測試集對訓練好的模型進行評估，通過 Precision 和 Recall 來衡量模型的預測，並分析其在不同資源使用區間的表現。

我們參考 Twitter [12] 在 Cost Forecasting 研究的結論，採用 XGBoost 分類模型與 TF-IDF 向量化提取 SQL 語句特徵，訓練精準率達 97% 以上，針對 CPU Time 和 Memory 分別構建預測模型；CPU Time 分為 [0-30s]、[30-300s]、[300-1800s]、[1800s+]，記憶體分為 [0-100MB]、[100-500MB]、[500-1000MB]、[1000MB+]。此外，我們也測試了 RandomForestClassifier 與 LogisticRegressionClassifier，但兩者在驗證集上的精準率與召回率均低於 XGBoost，且在處理高維稀疏特徵時的表現不如 XGBoost 穩定，因此最終選用 XGBoost 作為目標模型，根據特徵重要性（如 JOIN 數量、聚合函數數量）構建決策樹，預測資源需求區間。模型透過超參數調教最佳化預測的精準度。訓練結果如表 8 以及表 9 所示。

表 8：CPU Time 預測模型的每個分類的準確率以及召回率

| CPU Time (s)  | Precision | Recall |
|---------------|-----------|--------|
| [ 0, 30 ]     | 0.99      | 1.00   |
| [ 30, 300 ]   | 0.98      | 0.99   |
| [ 300, 1800 ] | 1.00      | 0.99   |
| [ 1800, ]     | 1.00      | 1.00   |

表 9：Memory 預測模型的每個分類的準確率以及召回率

| Peak Memory (MB) | Precision | Recall |
|------------------|-----------|--------|
| [ 0, 100 ]       | 0.99      | 1.00   |
| [ 100, 500 ]     | 0.97      | 0.99   |
| [ 500, 1000 ]    | 0.97      | 0.97   |
| [ 1000, ]        | 1.00      | 0.97   |

在資料集的部分，我們假設資料表大小固定（員工表約 100 萬行，10 欄；部門表約 1,000 行，5 欄），聚焦於 SQL 查詢的操作類型（如 SELECT、JOIN、GROUP BY）對資源需求的影響。固定資料表大小減少資料量變動的干擾，使模型能專注於學習操作類型的影響。例如，JOIN 數量增加導致數據匹配成本上升，放大 CPU Time

和記憶體需求；GROUP BY 和聚合函數因分組計算而增加記憶體使用量。實際環境中，資料表大小對記憶體需求有顯著影響。例如，JOIN 操作在 1 億行資料表上需更大記憶體儲存中間結果集，GROUP BY 有機會觸發數據溢出至磁盤，影響 CPU Time 和記憶體使用量。

### 4-3-3 訓練集與測試集的相似性分析

為評估模型的泛化能力，本研究將總計 7,000 + 筆查詢數據，隨機將資料分割為 80% 訓練集（6,000 + 筆）與 20 % 測試集（1,000 + 筆）。其為了確保查詢語句的唯一性，測試集中的查詢（如特定的 WHERE 條件或表名）不與訓練集重複。

在測試集的設計上，我們盡可能模擬真實的查詢場景，涵蓋不同複雜度的查詢類型。其中，簡單查詢約佔 15%，多為單表 SELECT 或簡單 WHERE 過濾；中等複雜查詢約佔 40%，通常包含單表或雙表 JOIN，並結合 WHERE、GROUP BY 等語法；高複雜度查詢則佔 45%，涵蓋多表 JOIN、子查詢或 CTE 等結構。此外，測試集中約有 5% 的查詢包含在訓練集中未完全出現的語法，例如多層嵌套子查詢、窗口函數 RANK() 等。此種設計旨在對模型的泛化能力進行壓力測試，以檢驗其在面對未見過的查詢時的適應性與預測準確度。

雖然測試集與訓練集在具體查詢內容上有所差異，但在高維度的語法特徵分佈上仍保持一致，例如 JOIN、GROUP BY 等常見語法的使用頻率相近，使得模型能掌握企業級資料庫的典型查詢模式。模型的泛化能力主要體現在對測試集中罕見語法的處理能力上。即使模型從未見過某個複雜的 CTE 結構，其依然能基於對 WITH 關鍵字、嵌套層次等特徵的學習，準確預測其資源需求，顯示出其能夠從已知語法組件推斷未知組合的能力。

### 4-3-4 Data Service

presto-query-predictor 模組基於 Flask 的網絡應用程序，用於提供模型接口服務。此應用程序可以將訓練好的模型部署為一個可供訪問的 API 服務。以下為 2 支



REST API 的資訊，其會接收 HTTP 請求，request body 中使用 JSON 格式將 query 欄位帶入查詢語句，其會返回 CPU Time 以及 Memory 的範圍。

以下是 Request 的示例 (包含 CPU Time 跟 Memory)

```
{
  "query": "with customer_total_return as\n(select sr_customer_sk as ctr_custome\nr_sk\n,sr_store_sk as ctr_store_sk\n,sum(SR_FEE) as ctr_total_return\nfrom store_\nreturns\n,date_dim\nwhere sr_returned_date_sk = d_date_sk\nand d_year =2000\ngrou\np by sr_customer_sk\n,sr_store_sk)\n select  c_customer_id\nfrom customer_total_r\neturn ctr1\n,store\n,customer\nwhere ctr1.ctr_total_return > (select avg(ctr_tota\nl_return)*1.2\nfrom customer_total_return ctr2\nwhere ctr1.ctr_store_sk = ctr2.ct\nr_store_sk)\nand s_store_sk = ctr1.ctr_store_sk\nand s_state = 'TN'\nand ctr1.ctr\n_customer_sk = c_customer_sk\norder by c_customer_id\nlimit 100"
}
```

以下是 Response 的示例

- /v1/cpu

```
{
  "cpu_pred_label": 0,
  "cpu_pred_str": "< 30s"
}
```

- /v1/memory

```
{
  "memory_pred_label": 0,
  "memory_pred_str": "< 1MB"
}
```

## 4-4 Infrastructure Provisioning Service Component

Infrastructure Provisioning Service Component (IPSC) 是本研究中負責動態調整 Presto 資源的元件。由於 Presto Worker 部署在 Oracle Cloud Infrastructure (OCI) 的虛擬機實例 (VM Instance) 上，每個 Presto Worker 依賴於單獨的 VM Instance 運行，IPSC 通過對 VM Instance 的管理，實現 Presto Worker 數量的調整，以滿足查詢負載的需求變化。

### 4-4-2 運作機制與決策邏輯

IPSC 與 Resource Estimation Service Component (RESC) 協作，接收其推估的 Presto worker 數量，並以此為基礎調整 OCI 上的 VM Instance 數量。相較於通用 Auto Scaling [9]，IPSC 整合查詢預測與 Infrastructure as Code (IaC) 進行資源配置。

在運作當中，IPSC 會比較當前運行的 VM Instance 數量（即當前可用的 Presto worker 數量）與 RESC 估計的需求，如圖 12 的決策邏輯流程圖所示，其根據以下三種情況執行從接收工作節點的數量評估到最終資源調整的對應操作：

1. Presto Worker 數量足夠使用

若當前運行的 VM Instance 數量滿足 RESC 估算的 Presto Worker 數量，IPSC 將保持現有配置不變。

2. Presto Worker 數量超過當前所需

若當前運行的 VM Instance 數量超出 RESC 評估的需求，IPSC 會停止多餘的 VM Instance，減少 Presto Worker 數量。選擇停止而非刪除 VM Instance 的原因在於，大多數查詢在默認 3 個 Worker 的配置下需要擴展。因此，相較於創建與刪除操作，啟動與停止操作的效率較高，因為後續若需再次增加 VM Instance，啟動已停止的 Instance 比創建新實例更快。

3. Presto Worker 數量不足使用

若當前運行的 VM Instance 數量不足以滿足 RESC 估算的 Presto Worker 數量，IPSC 會進一步判斷 OCI 上處於停止狀態的 VM 數量

- 若停止狀態的 VM Instance 數量足以補足需求，IPSC 會啟動對應數量的停止 VM Instance，使其重新啟動 Presto Worker。
- 若停止狀態的 VM Instance 數量不足，IPSC 會先啟動所有可用的停止 VM Instance，並創建新的 VM Instance 以補足差額，並在這些 VM Instance 上的 Presto Worker。

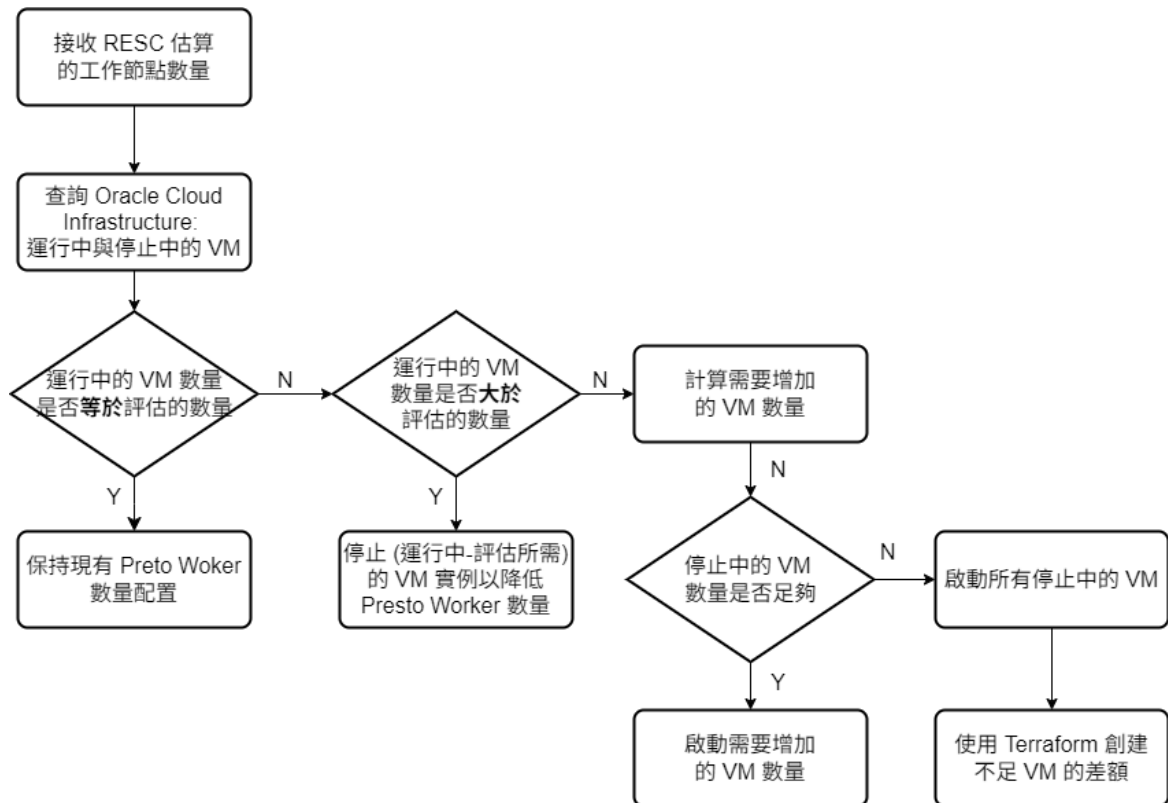


圖 12：IPSC 決策邏輯流程圖

#### 4-4-3 技術整合

IPSC 透過以下技術實現對 Oracle Cloud Infrastructure 虛擬機實例 (VM Instance) 的管理，以利對於 Presto Worker 數量的調整：

- OCI Python SDK

IPSC 使用 OCI Python SDK 獲取與操作 VM Instance 的狀態，如圖 13 示例代碼中所撰寫，啟動、停止或獲取 VM Instance 狀態。

```
def control_instance(compute_client, instance_id, action):
    try:
        print(f"Attempting to {action} instance: {instance_id}")
        response = compute_client.instance_action(instance_id=instance_id, action=action)
        print(f"Action '{action}' initiated. Status code: {response.status}")

        target_state = "RUNNING" if action == "START" else "STOPPED"
        get_instance_response = compute_client.get_instance(instance_id)
        waiter_result = oci.wait_until(
            compute_client,
            get_instance_response,
            'lifecycle_state',
            target_state,
            max_wait_seconds=300
        )
```

圖 13：控制 OCI VM Instance 的啟動/停止代碼示例

```
def get_instance_status(compute_client, instance_id):
    try:
        instance = compute_client.get_instance(instance_id).data
        print(f"Instance '{instance.display_name}' status: {instance.lifecycle_state}")
    except Exception as e:
        print(f"[ERROR] Failed to get instance status: {e}", file=sys.stderr)
```

圖 14：獲取 OCI VM Instance 的狀態代碼示例

- Terraform

IPSC 利用 Terraform 進行 VM Instance 的創建與刪除操作，確保新創建的 VM Instance 其部署符合 Presto Worker 的運行需求。作為 IaC 工具，如圖 15 所示，Terraform 以聲明式方式管理 OCI 資源，確保配置的一致性與複用性。

```

variable "instances" {
  type = map(object({
    display_name = string
    shape        = string
    ocpus        = string
    memory_in_gbs = string
  }))
  default = {
    instance1 = {
      display_name = "vm-presto-worker04"
      shape        = "VM.Standard.E5.Flex"
      ocpus        = "8"
      memory_in_gbs = "20"
    }
    instance2 = {
      display_name = "vm-presto-worker05"
      shape        = "VM.Standard.E5.Flex"
      ocpus        = "8"
      memory_in_gbs = "20"
    }
    ...
  }
}

# 使用 for_each 來創建多個實例
resource "oci_core_instance" "presto_vms" {
  for_each = var.instances

  availability_domain = data.oci_identity_availability_domains.ads.availability_domains[0].name
  compartment_id      = var.compartment_ocid
  shape               = each.value.shape
  shape_config {
    memory_in_gbs = each.value.memory_in_gbs
    ocpus         = each.value.ocpus
  }
  ...
}

```

圖 15：Terraform 資源聲明代碼示例

## 4-5 Workflow Management System

現行若要從用戶端發送 SQL 語句並根據查詢負載動態調整 Presto 的運算資源以實現 Auto Scaling 機制，傳統方法往往需要手動整合多個異質模組與工具，例如：查詢預測模型、資源調度模組及基礎設施部署工具。此做法不僅系統耦合性高、維護成本增加，且缺乏統一的工作流程編排與執行狀態追蹤能力，進而影響系統的可靠性與

可維護性。為解決這些問題，我們在系統中引入 Airflow 作為系統的工作流程管理中樞，透過 DAG 整合查詢負載預測與資源配置的工作流程，實現 Presto Auto Scaling 機制。如 2-3 章節的圖 3 所示，Airflow 是一個以有向無環圖（DAG）為核心的工作流程管理系統，具備動態任務排程、跨任務資料傳遞以及錯誤回復功能。其設計能夠有效協調複雜且具階段性的資料處理流程，並與外部系統實現高度整合。Airflow 的 DAG 結構允許明確定義任務之間的依賴關係，確保任務按順序執行，同時提供即時監控與日誌記錄功能，從而提升系統的可靠性以及可維護性。

在本研究中，我們設計一個端到端的 DAG，用以管理 Presto Auto Scaling 的完整流程。該 DAG 包含四個任務，其邏輯順序與功能設計如下所述：

#### 1. 任務一 (T1)：調用 Query Prediction Service Component

如圖 15 代碼示例，T1 啟動時，DAG 從用戶端的 HTTP 請求中提取 SQL 語句，並將其作為輸入參數，通過 REST API 調用 Query Prediction Service Component。該元件如 4-3 章節所述，利用模型來預測查詢執行所需的 CPU Time 與 Memory，並將預測結果語 SQL 語句以 JSON 格式返回，再將數據透過 XCom 傳遞至後續任務。

```
def query_prediction_workflow(**context):
    """整合 workflow：獲取 SQL 查詢並預測資源需求"""
    # 步驟1：獲取 SQL 查詢
    sql_query = get_sql_from_dagrun(**context)

    # 步驟2：預測資源需求
    prediction = predict_query_resources(**context)

    # 返回完整結果供後續任務使用
    return {
        'sql_query': sql_query,
        'cpu_prediction': prediction['cpu_time'],
        'memory_prediction': prediction['peak_memory']
    }
```

圖 16：DAG 任務一代碼示例

## 2. 任務二 (T2)：資源推估與 Auto Scaling 決策

DAG 從 XCom 中獲取 Query Prediction Service Component 傳遞的資源預測數據，並將其作為輸入傳遞至 Resource Estimation Service Component。如 4-2 章節所述，該元件結合模型預測的結果與歷史查詢分析資料，評估執行當前查詢所需的 Presto Worker 數量。其會計算滿足查詢效能需求的 Worker 數量，並與當前叢集配置進行比較，決定是否需要擴展或縮減 Worker。最後將估算結果透過 XCom 傳遞至下一任務，以供資源調整。

```
# 導入資源估算模組
from presto_resource_estimator import ResourceEstimator

# 創建估算器實例
estimator = ResourceEstimator()

# 執行估算
required_workers = estimator.estimate_workers(
    cpu_time=predicted_cpu_time,
    peak_memory=predicted_peak_memory
```

圖 17：DAG 任務二代碼示例

## 3. 任務三 (T3)：基礎設施配置

此任務負責執行 Infrastructure Provisioning Service Component，該元件接收來自 T2 的目標 Worker 數量，並如 4-4 章節所述，其會根據當前 OCI 上的 VM Instance 狀態，制定相應的資源調整策略。

```
# 從 XCom 獲取所需 worker 數量
target_workers = ti.xcom_pull(key='required_workers', task_ids='estimate_workers')
...
# 確定是擴展還是縮減
if target_workers > current_running_count:
    # 需要擴展 - 首先嘗試啟動已停止的實例
    ...
    # 優先啟動已停止的實例
    for i, inst in enumerate(stopped_workers):
        if i >= workers_to_add:
            break
```

圖 18：DAG 任務三代碼示例

#### 4. 任務四：執行 SQL 查詢

在 Presto 叢集完成資源調整之後，將用戶提交的 SQL 語句派發至 Presto 叢集進行執行，確保查詢在調整後的資源配置下運行。

透過上述設計，我們整合端到端的工作流程，實現從查詢預測、資源估算到基礎設施調整的 Presto Auto Scaling 機制。此外，這種工作流程管理方式為資料的查詢與動態資源管理奠定基礎，並為後續研究提供對應的參考架構。



## 五、 研究結果

### 5-1 實驗環境說明

為驗證本研究所提出之方法的可行性與實際效益，實驗環境部署於 Oracle Cloud Infrastructure (OCI) 之虛擬機實例 (VM Instance) 上，並建置 Presto 分散式查詢引擎作為查詢執行平台，使用 TPC-DS 作為測試數據集。整體架構結合 Apache Airflow 進行自動化工作流程管理，針對 Presto Auto Scaling 機制進行驗證，環境配置如下：

- Presto：版本為 0.291，執行於配置 8 vCPU 的 AMD VM.Standard.E5.Flex 虛擬機器，搭配 20GB 記憶體，作業系統為 Ubuntu 20.04。
- Apache Airflow：版本為 2.10.2，負責排程與觸發查詢任務。
- Terraform：版本為 v1.9.6，所使用的 OCI Provider 版本為 v6.31.0，負責動態創建/刪除 Presto 節點資源。
- 數據集：實驗採用標準化 TPC-DS Benchmark [14] 作為測試資料集，共計 24 張資料表，總行數約為 6 千萬行（詳細資料如表 10 所示）。
- 查詢工作負載：採用 TPC-DS 所定義之 99 條 SQL 查詢語句，涵蓋四種類型：連接查詢、聚合查詢、巢狀子查詢，以及窗口函數查詢，以模擬多樣化商業分析場景下的查詢行為。

表 10：TPC-DS 資料集

| #  | 表名                     | 行數         | 大小      | 備註       |
|----|------------------------|------------|---------|----------|
| 1  | call_center            | 30         | ~5 KB   | 服務中心     |
| 2  | catalog_page           | 21,000     | ~2.5 MB | 郵寄目錄產品頁  |
| 3  | catalog_returns        | 1,440,000  | ~260 MB | 郵寄目錄銷售退回 |
| 4  | catalog_sales          | 14,400,000 | ~2.6 GB | 郵寄目錄銷售   |
| 5  | customer               | 2,000,000  | ~310 MB | 客戶主檔     |
| 6  | customer_address       | 1,000,000  | ~150 MB | 客戶地址     |
| 7  | customer_demographics  | 1,920,800  | ~280 MB | 客戶地理區域   |
| 8  | date_dim               | 73,049     | ~15 MB  | 日期維度     |
| 9  | household_demographics | 7,200      | ~1 MB   | 家戶地理區域   |
| 10 | income_band            | 20         | ~2 KB   | 收入分群     |

|    |               |            |         |        |
|----|---------------|------------|---------|--------|
| 11 | inventory     | 4,000,000  | ~680 MB | 存貨     |
| 12 | item          | 180,000    | ~32 MB  | 商品品項   |
| 13 | promotion     | 300        | ~50 KB  | 促銷     |
| 14 | reason        | 65         | ~10 KB  | 季節維度   |
| 15 | ship_mode     | 20         | ~5 KB   | 出貨模式   |
| 16 | store         | 402        | ~60 KB  | 商店     |
| 17 | store_returns | 2,880,000  | ~520 MB | 商店銷售退回 |
| 18 | store_sales   | 28,800,000 | ~5.2 GB | 商店銷售   |
| 19 | time_dim      | 86,400     | ~12 MB  | 時間維度   |
| 20 | warehouse     | 20         | ~4 KB   | 倉庫     |
| 21 | web_page      | 60         | ~9 KB   | 銷售網頁   |
| 22 | web_returns   | 720,000    | ~130 MB | 網路銷售退回 |
| 23 | web_sales     | 7,200,000  | ~1.3 GB | 網路銷售   |
| 24 | web_site      | 30         | ~4 KB   | 銷售網站   |

## 5-2 測試場景與對比方法

為驗證本研究所提出之 Presto Auto Scaling 機制的效益，我們設計兩種對比實驗場景，分別為靜態資源配置與本研究之 Auto Scaling 機制：

1. **靜態配置場景：**Presto 叢集配置為 1 個 Coordinator 搭配 3 個 Workers；在此場景中，不論查詢類型或資源需求，Worker 數量於整個實驗期間皆不變動。
2. **Auto Scaling 場景：**本研究所提出的動態資源調整方法，根據要執行查詢的 SQL 語句整合 QPSC、RESC、IPSC 元件以調整 Worker 數量。此場景中，Worker 的動態調整會在查詢執行前完成，確保查詢過程中不受資源調整的延遲影響。本研究聚焦於資源配置與查詢效能的關聯，因此未將 Auto Scaling 所花費的時間納入查詢效能或成本計算之中。

為提升結果之代表性與穩定性，實驗對每一種場景皆重複執行 TPC-DS 所提供的 99 條 SQL 語句共 10 次，並計算各項修剪平均值。其 SQL 語句亦依據語法特性分為四種類型：連接查詢、聚合查詢、子查詢與窗口函數查詢，並進行個別類型與混合情境下的觀察與分析，四個類型的 SQL 語句說明如下：

- 連接查詢: 涉及多表 JOIN 操作，根據條件檢索資料。
- 聚合查詢: 執行統計計算，包含 SUM、AVG、COUNT 等函數。

- 子查詢：嵌套於主查詢中，一般作為條件或限制。
- 窗口函數查詢：在查詢結果中進行排名、分組、排序等操作。

本研究的實驗評估指標包含：

- **查詢延遲**：以單一查詢的執行時間（單位：秒）為測量指標，目的在評估不同資源配置對查詢處理效率的影響。執行時間從查詢發送至結果返回的總耗時計算，觀察不同資源配置對處理一個查詢需要花多少時間。
- **資源使用比較**：根據實驗的結果記錄查詢所使用的 Presto Worker 數量、CPU Time 與 Memory 使用量，分析資源的使用。
- **成本效益**：評估不同的配置在雲端環境中資源利用與經濟性之間的平衡表現，作為衡量系統性價比的標準，其反映整體的查詢處理效率與運算成本的關係。該指標以單位成本下每秒完成的查詢數（單位：查詢/秒/元，NTD）為測量標準，數值越高表示系統在相同成本下能更高效地完成查詢，反映更高的資源利用效率。

## 5-3 查詢延遲比較

### 5-3-1 指標說明

在查詢延遲的比較中，我們選擇 SQL 語句查詢執行時間（Execution Time）作為評估的指標。執行時間是指 SQL 語句在 Presto 叢集上從提交到完成所需的時長（以秒為單位）。此指標反映查詢的處理效率，對用戶體驗和系統響應速度具有影響。

### 5-3-2 實驗數據呈現

在表 11 顯示，Auto Scaling 機制在總執行時間縮短了 14%，四個分群其查詢執行時間減少範圍如表 12 則顯示，在 12% 到 42% 不等，其中本研究的 Auto Scaling 機制排除樣本數較少的連接查詢之外，其在聚合查詢上的查詢效能最佳，有將近 21% 的查詢執行時間節省。

表 11：靜態配置與 Auto Scaling 機制的總執行時間比較

| 靜態配置執行時間總和 (s) | Auto Scaling 執行時間總和 (s) | 總變化百分比  |
|----------------|-------------------------|---------|
| 4916.12        | 4186.52                 | -14.84% |

表 12：靜態配置與 Auto Scaling 機制下查詢分群的總執行時間比較

| 分群名稱   | 查詢數量 | 靜態配置執行時間總和 (秒) | Auto Scaling 執行時間總和 (秒) | 變化百分比    |
|--------|------|----------------|-------------------------|----------|
| 連接查詢   | 3    | 31.82          | 18.29                   | -42.49 % |
| 聚合查詢   | 28   | 802.39         | 634.17                  | -20.97 % |
| 子查詢    | 53   | 3501.48        | 3048.16                 | -12.94 % |
| 窗口函數查詢 | 15   | 580.43         | 485.9                   | -16.29 % |

再進一步往下看到圖 19 以及圖 20 所顯示在 Auto Scaling 機制下，四個查詢分組的執行時間降低比例以及查詢數量比較，99 筆查詢之中，有 92% 的查詢降低了查詢的執行時間，其中在子查詢表現最佳，有 94% 的查詢降低了執行時間，其次是窗口函數查詢，有 93 % 的查詢降低了查詢時間；而在連接查詢的部分，因為在 3 筆查詢總數中有 1 筆是增加了查詢時間，故其降低執行的比例為 66 %。

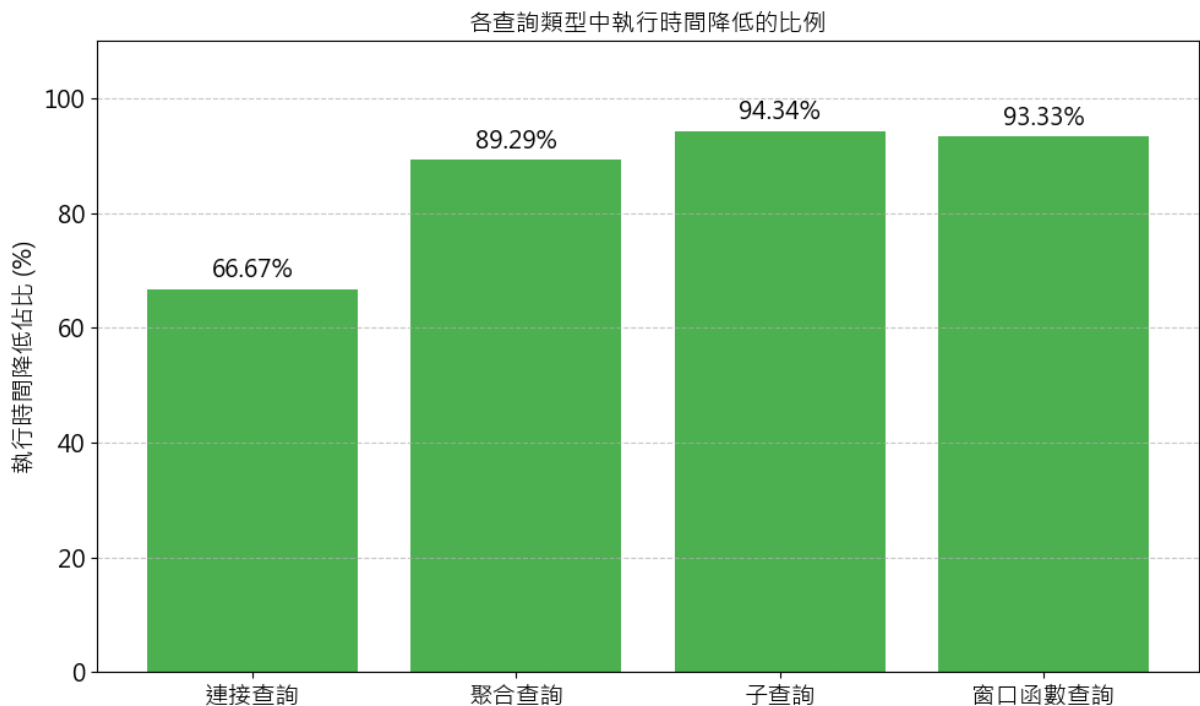


圖 19：Auto Scaling 機制下各類查詢類型執行時間降低比例

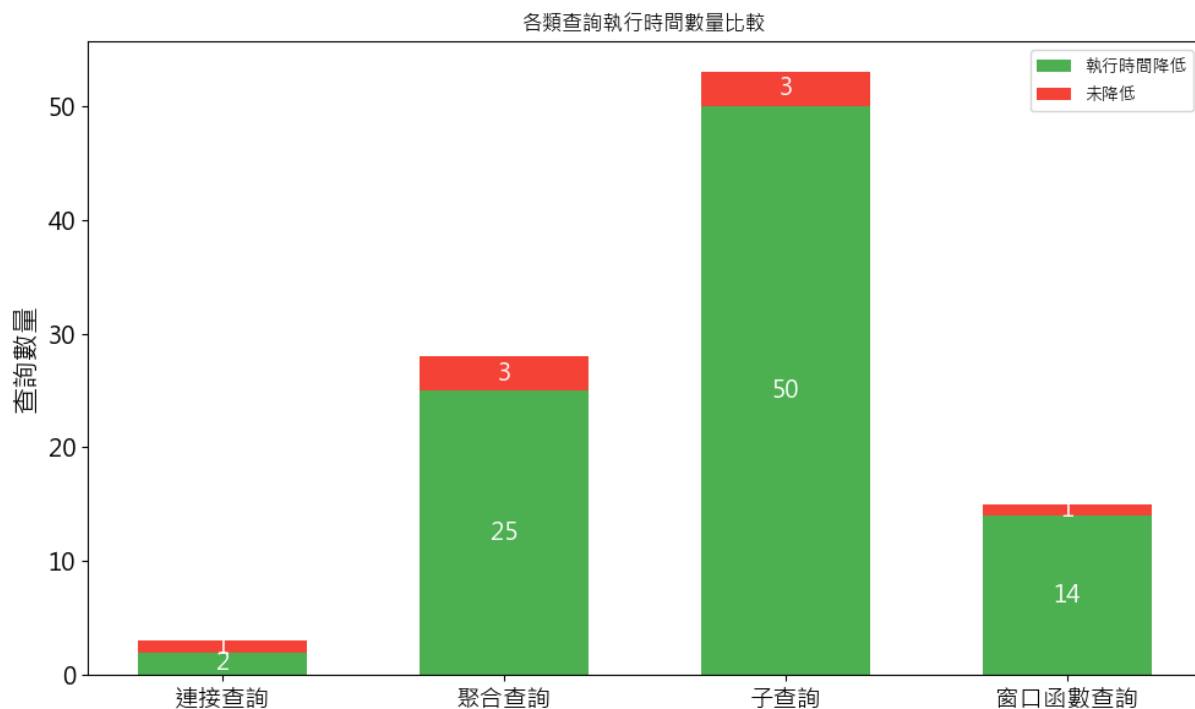


圖 20：Auto Scaling 機制下各類查詢執行時間數量比較

接著進行分群查詢的執行時間的分析，在圖 21 中連接查詢時間降低比例僅 66%，因查詢數量只有 3 筆，受節點數影響明顯。時間降低的查詢使用 4 個 Worker 執行查詢，其效率提升顯著；時間增加的查詢（如：Query84）則使用 2 個 Worker，低負載以致於效率下降。

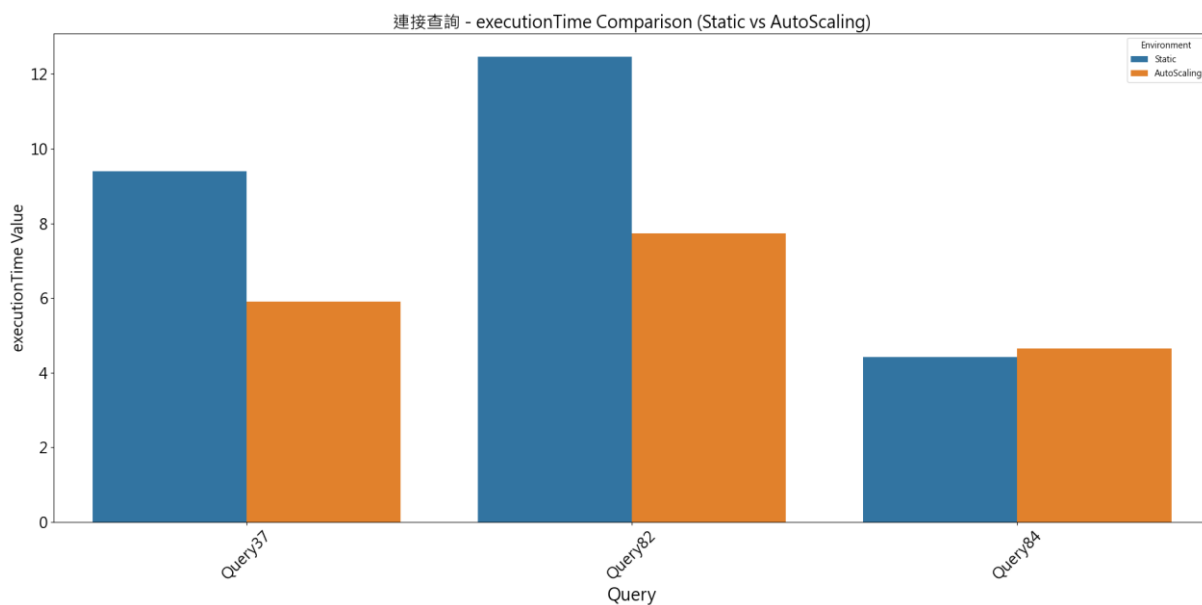


圖 21：連接查詢下 SQL 語句執行時間對比

如圖 22 所示，在子查詢時間降低比例最高，顯示 Auto Scaling 的 Worker 調整對子查詢的效率提升顯著。時間增加的查詢（3 筆）均為低負載查詢，節點數減少造成效率下降。時間降低的查詢中，有 35 筆查詢 Worker 數量為 5。

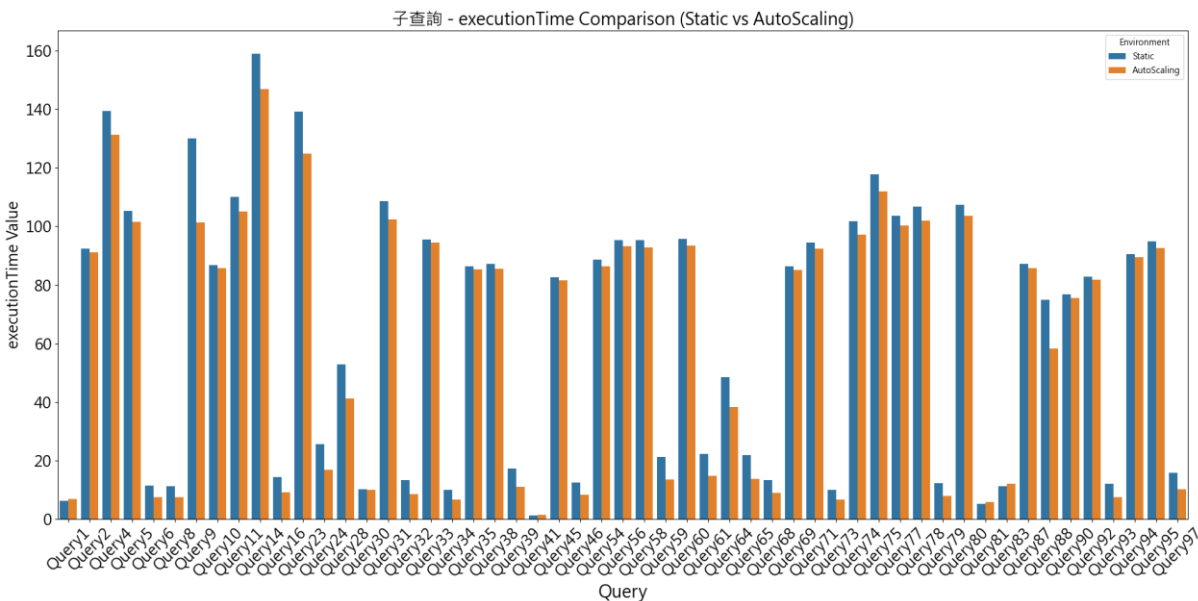


圖 22：子查詢下 SQL 語句執行時間對比

在圖 23 窗口函數查詢時間降低比例為次高，顯示動態的 Worker 調整（多數為 5）有效提升效率；時間增加的查詢為低負載，Worker 數量為 2 導致效率下降。

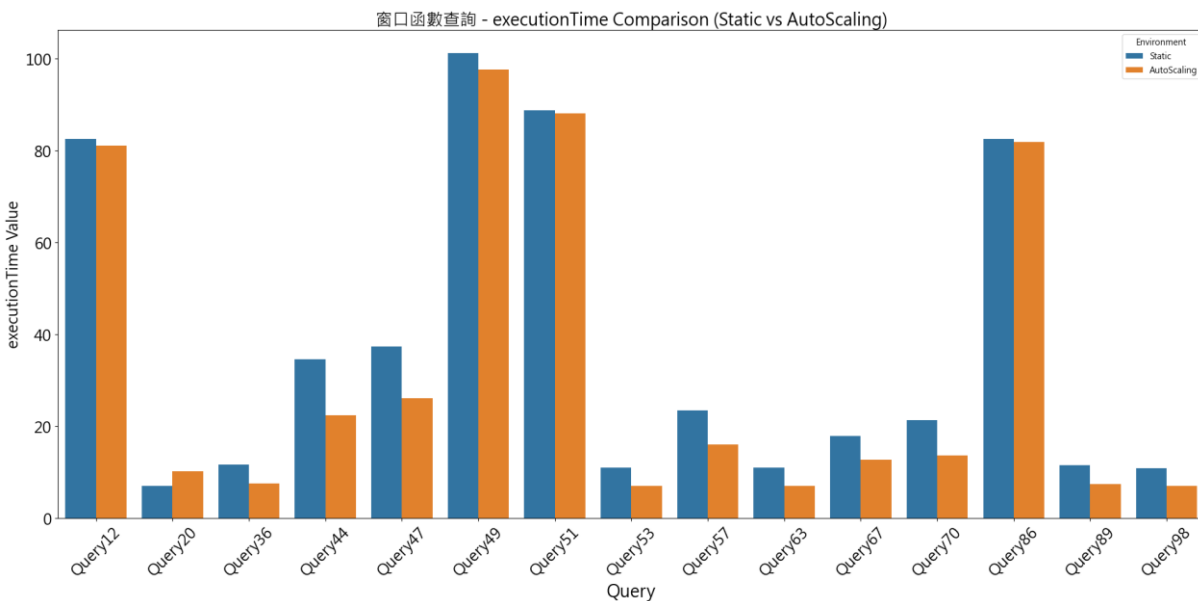


圖 23：窗口函數查詢下 SQL 語句執行時間對比

在聚合查詢時間降低比例為 89%，如圖 24 所示，Worker 數量為 5 的查詢有 22 筆，其查詢效率提升顯著；時間增加的 3 筆查詢均為低負載，Worker 數量為 2 以致效率下降。Worker 數量為 4 的有 3 筆查詢時間降幅大，如: Query72，Auto Scaling 後減少 39.12 秒。

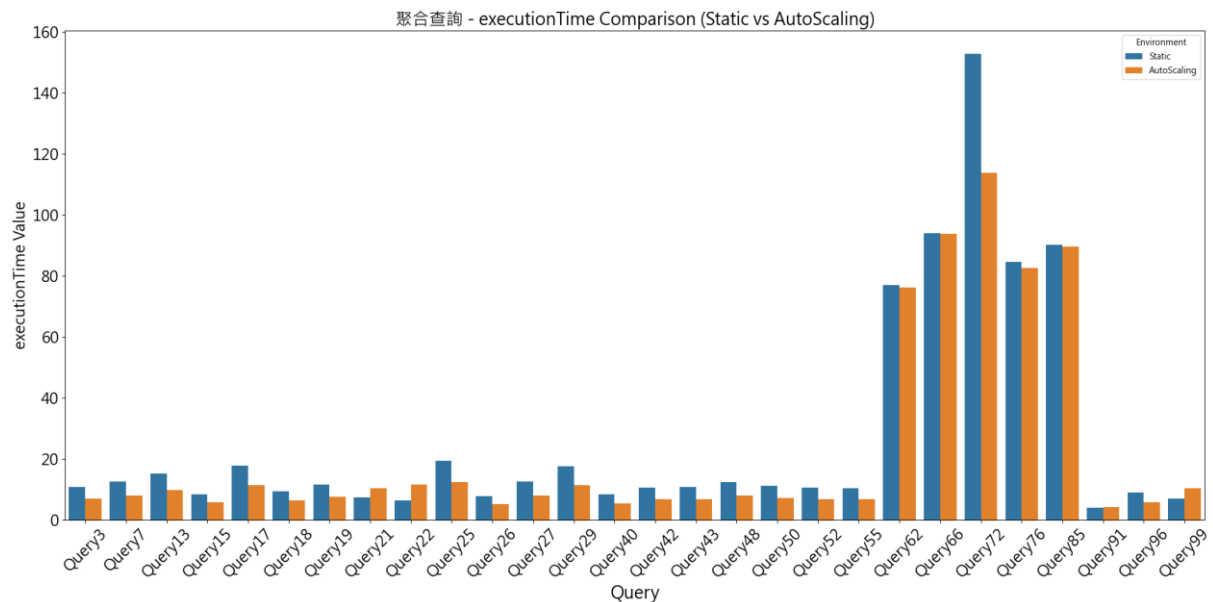


圖 24：聚合查詢下 SQL 語句執行時間對比

相較靜態配置的固定 1 個 Worker，本研究的 Auto Scaling 機制根據查詢需求動態調整節點數量，提升並行處理能力，縮短 TPC-DS 複雜查詢的執行時間。

## 5-4 資源使用比較

### 5-4-1 指標說明

在比較 Auto Scaling 機制與靜態配置下 Worker 節點的資源使用情況，評估動態資源分配在資源效率和查詢效能方面的優勢與潛在挑戰。因此我們於 Worker 數量、CPU Time 以及 Memory 的使用，並分析這些指標在不同查詢類型中的表現差異。

### 5-4-2 Worker 節點資源使用數據展示

Auto Scaling 增加 80% 查詢的 Worker 數量（表 12），76% 的查詢提升了 CPU Time（表 14）。記憶體增加 69%（表 15），因節點間資料交換開銷。

### 5-4-2-1 Worker 節點數量調整分析

從表 13 以及表 14 的比較數據中我們察覺到，80% 的查詢在 Auto Scaling 機制下增加了 Worker 節點，顯示大多數查詢的資源需求超出靜態配置的容量。

表 13：Worker 數量變化比較

| Worker 變化 | 調整資源的查詢 | 占比     |
|-----------|---------|--------|
| 增加 Worker | 80      | 80.81% |
| 減少 Worker | 19      | 19.19% |

依據查詢類型分組的詳細調整情況如下表 13 所示：

表 14：分群 Worker 數量比較

| 群組名稱   | 查詢數量 | 增加 Worker 的查詢數量 | 增加占比   | 減少 Worker 的查詢數量 | 減少占比   |
|--------|------|-----------------|--------|-----------------|--------|
| 連接查詢   | 3    | 2               | 66.67% | 1               | 33.33% |
| 聚合查詢   | 28   | 22              | 78.57% | 6               | 21.43% |
| 子查詢    | 53   | 44              | 83.02% | 9               | 16.9%  |
| 窗口函數查詢 | 15   | 12              | 80%    | 3               | 20%    |

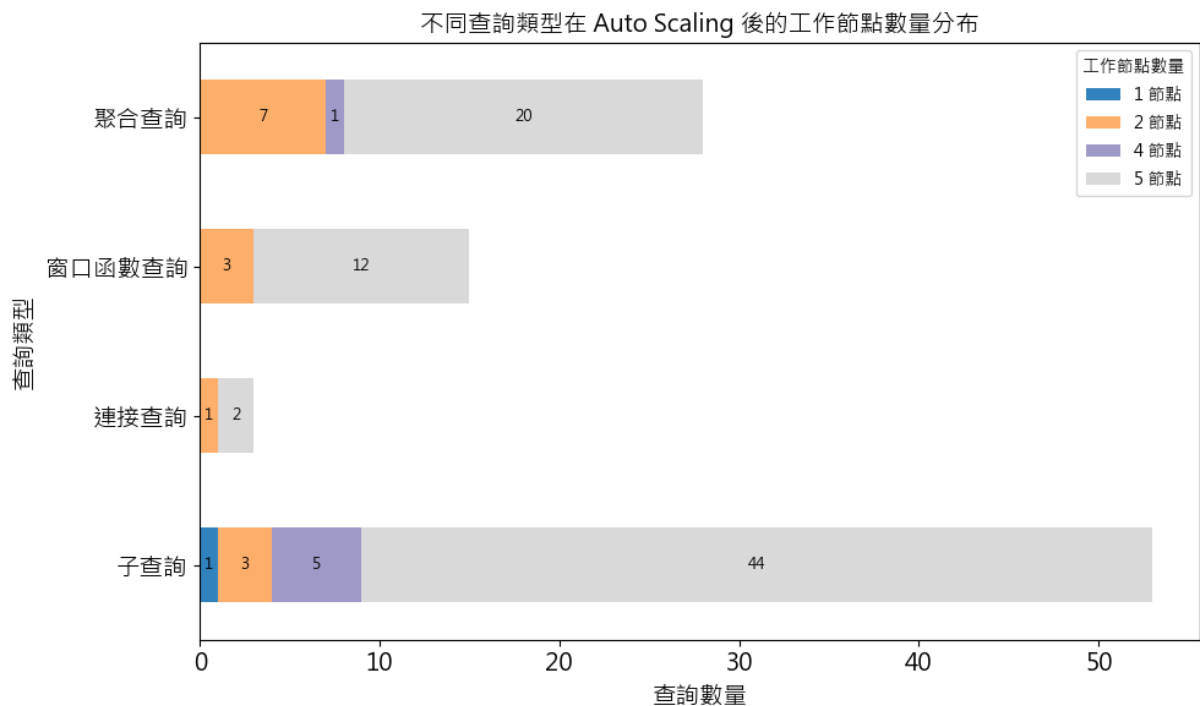


圖 25：Auto Scaling 後分群 Worker 數量分布



Auto Scaling 傾向於在子查詢和窗口函數查詢中分配更多工作節點，如圖 25 所示，增加比例分別達 83% 和 80%，顯示其根據查詢複雜度動態調整資源的特性，對於這類查詢對資源的高需求。在聚合查詢的 78%，顯示其相較其他類型的查詢資源需求的變異性較大，而連接查詢則因查詢數量較少的原因，其在增加的占比略低。

#### 5-4-2-2 CPU Time 分析

在表 15 中的比較數據顯示，增加 Worker 後有 76% 的查詢在 CPU Time 也增加；其中子查詢中有 81%，顯示此類型查詢普遍在資源調整後 CPU Time 提升了。然而透過 Auto Scaling 增加 Worker 數量的查詢，其 CPU Time 上升是必然的結果，因為此會透過增加併發度，降低延遲、提高吞吐量，但會換取更高的資源使用量。因此以下會在各個分群中查詢中，選擇增幅與降幅最大的查詢進行分析比較。

表 15：Worker 數量與 CPU Time 的比較

| 群組名稱   | 查詢數量 | Worker 與 CPU Time 增加 | Worker 與 CPU Time 減少 | Worker 增加 CPU Time 減少 | Worker 減少 CPU Time 增加 |
|--------|------|----------------------|----------------------|-----------------------|-----------------------|
| 連接查詢   | 3    | 2                    | 1                    | 0                     | 0                     |
| 聚合查詢   | 28   | 21                   | 5                    | 2                     | 0                     |
| 子查詢    | 53   | 43                   | 9                    | 1                     | 0                     |
| 窗口函數查詢 | 15   | 10                   | 3                    | 2                     | 0                     |

子查詢當中最大的 CPU Time 增幅與降幅為 Query 14 以及 Query39；如表 16 所示，Query 14 因使用多層 CTE、複雜 JOIN 與資源密集操作導致擴展性受限且 CPU 負荷增加，而 Query 39 透過統計分析與自 JOIN 優化，在分散式環境中有效分攤計算壓力並降低資源消耗，展現更高的併發擴展效率。

表 16：Query 14 與 Query 39 語句分析

| Query    | JOIN 數 | JOIN 類型    | 聚合操作           | ORDER BY | 條件複雜度              |
|----------|--------|------------|----------------|----------|--------------------|
| Query 14 | 0      | N/A        | 聚合、CASE、HAVING | 是        | 有 UNIONS、HAVING 條件 |
| Query 39 | 5      | inner join | 多聚合、CASE 條件    | 是        | 條件中等、JOIN 多        |

在表 17 聚合查詢分析中，Query 22 與 Query 72 為 CPU Time 增幅與降幅最大的查詢；Query 22 因單調的 ROLLUP 聚合與有限分區導致擴展反效果（節點減少反而提升效率），而 Query 72 透過多個 Worker 分工有效消化複雜度較高的 JOIN 與跨表條件，展現其越複雜的查詢，越能受惠於自動擴展的分散式運算特性。

表 17：Query 22 與 Query 72 語句分析

| Query    | JOIN 數 | JOIN 類型    | 聚合操作               | ORDER BY | 條件複雜度  |
|----------|--------|------------|--------------------|----------|--------|
| Query 22 | 2      | inner join | GROUP BY<br>ROLLUP | 是        | 條件簡單   |
| Query 72 | 9      | outer join | 聚合、CASE、<br>COUNT  | 是        | 條件多且複雜 |

#### 5-4-2-3 Memory 分析

在表 18 中所呈現在增加 Worker 後，Memory 的使用普遍上升，尤其在子查詢和聚合查詢中較為明顯。起因為 Worker 之間的資料傳遞所致。根據 Presto 官方文檔 [3]，分散式查詢引擎在執行複雜查詢時，Worker 之間的大量資料交換會提升 Memory 的佔用。下面針對不同分群中，記憶體使用情況進行分析。

表 18：Worker 數量與總記憶體峰值的比較

| 群組名稱   | 查詢數量 | Worker 與<br>Memory 增<br>加 | Worker 與<br>Memory 減<br>少 | Worker 增<br>加 Memory<br>減少 | Worker 減<br>少 Memory<br>增加 |
|--------|------|---------------------------|---------------------------|----------------------------|----------------------------|
| 連接查詢   | 3    | 1                         | 1                         | 1                          | 0                          |
| 聚合查詢   | 28   | 20                        | 5                         | 2                          | 1                          |
| 子查詢    | 53   | 37                        | 5                         | 7                          | 4                          |
| 窗口函數查詢 | 15   | 6                         | 3                         | 6                          | 0                          |

表 19：Query 39 與 Query95 語句分析

| 項目       | Query 39                    | Query 95                     |
|----------|-----------------------------|------------------------------|
| 子查詢數量    | 1 個 CTE（子查詢）、self-join 兩次   | 2 個子查詢                       |
| 聚合       | CTE、self-join 比對相鄰月份        | count(distinct), sum()       |
| JOIN 數   | 4 個 inner join、self-join 一次 | 4 個 join                     |
| ORDER BY | 輸出結果需排序大量欄位                 | 排序 count(distinct)、limit 100 |
| 資料複製量    | CTE、self join               | in 子查詢限制                     |
| 中間結果儲存   | 高                           | 中等偏低                         |

在表 19 中，Query 39 因複雜聚合與數據膨脹在節點擴展時增加了記憶體的使用，而 Query 95 透過過濾下推與結果截斷使分散式架構能有效分攤記憶體壓力，體現執行計劃設計對分散式資源消耗的關鍵影響。

表 20：Query 72 與 Query 99 語句分析

| 項目          | Query 72                          | Query 99          |
|-------------|-----------------------------------|-------------------|
| 資料來源表數      | 9 個表                              | 5 個表              |
| Join 類型     | 多個 INNER JOIN、2 個 LEFT OUTER JOIN | 多個 INNER JOIN     |
| Join 條件     | 複雜                                | 單純                |
| GROUP BY 欄位 | 欄位多                               | 欄位少               |
| 條件篩選        | 多                                 | 單一                |
| 聚合函數        | SUM(CASE WHEN...), COUNT(*)       | SUM(CASE WHEN...) |
| ORDER BY    | 多欄位排序                             | 較少欄位排序、LIMIT 100  |
| 中間表大小       | 結合量大                              | 記憶體用量可控           |
| 輸出欄位多寡      | 欄位多、結構複雜                          | 欄位較少、結構單純         |

如表 20 所呈現，Query 72 因非等值 JOIN 與高基數聚合在分散式擴展時加劇記憶體碎片化，而 Query 99 透過結構化主表連接與結果截斷優化，在集中運算中降低記憶體消耗，體現查詢複雜度與分散式擴展策略對記憶體效能的雙向影響。

### 5-4-3 比較分析

本研究所提出的 Auto Scaling 機制能根據查詢需求動態調整 Worker 節點數量，80% 的查詢需增加節點，顯示靜態配置在資源供給上有可以擴容的空間。子查詢和窗口函數查詢的高增加比例（83% 和 80%）顯示這兩種類型的查詢對資源的依賴度較高。在 CPU 方面，Auto Scaling 機制雖提升了大多數查詢的總 CPU Time，其透過提升 CPU 的使用獲得更大的並發數以及處理效能。而 Memory 部分則是在增加 Worker 後，總記憶體峰值普遍上升，子查詢和聚合查詢較為顯著。此反映分散式系統中節點間協作的記憶體開銷，需進一步優化以平衡效能與資源使用。

上述驗證 Auto Scaling 在資源效率和查詢效能上的優勢，尤其在高資源需求的查詢中表現突出。然而，記憶體使用的增加顯示其挑戰性。後續研究可優化記憶體分配策略，並進一步評估雲端成本影響。

## 5-5 成本效益比較

### 5-5-1 成本效益指標定義

在雲端運算環境中，成本效益是衡量系統效能與資源管理效率的關鍵指標，用以評估單位成本下查詢處理的效率。因此，本研究比較靜態配置與 Auto-Scaling 機制在成本效益上的表現，探討 Auto-Scaling 如何在查詢執行時間與 Oracle Cloud Infrastructure (OCI) 虛擬機實例 (VM Instance) 使用的成本之間實現平衡，並驗證其在資源分配上的優勢。成本效益指標以單位成本下每秒完成的查詢數為測量標準，單位為「查詢/秒/元 (新台幣)」。數值越高，表示其在相同成本下能更高效地處理查詢，反映更高的資源利用效率。採用此指標的理由在於，雲端運算環境中資源使用與成本密切相關，靜態配置可能導致資源浪費或效能不足，而 Auto Scaling 需在提升效能的同時控制成本增長。以下進行成本效益的推導計算說明：

- 公式一：單一查詢執行成本

單一查詢的執行成本 (*Unit Query Cost*，單位：NTD) 由查詢執行時間、OCI VM 單價及所使用的 Presto 節點數量共同決定。查詢執行時間 ( $T$ ，單位：秒) 涵蓋查詢從 Planning、Scheduling、Running 的全流程耗時。單個 VM Instance 的單價基於按需定價模型，根據 Oracle Estimator，VM.Standard.E5.Flex 運算型態 (8 OCPU、20 GB 記憶體) 的單價為 8.74 NTD/小時，約為 0.002428 NTD/秒，該值作為 *VM Unit Price* 的參考，假設使用  $n$  個 Presto 節點 (包括 Coordinator 與 Worker)，則單一查詢執行成本計算如下。

$$Unit\ Query\ Cost = T \times VM\ Unit\ Price \times Presto\ Nodes$$

- 公式二：總查詢執行成本

針對多筆查詢，總查詢執行成本 (Total Query Cost，單位：元) 為每筆查詢執行成本的加總，計算公式如下，其中， $N$  為查詢總數，在本研究中其值為 99 (筆)， $Unit\ Query\ Cost(i)$  為第  $i$  筆查詢的執行成本，依據公式一計算。

$$Total\ Query\ Cost = \sum_{i=1}^N Unit\ Query\ Cost(i)$$

● 公式三：單一查詢的成本效益

單一查詢的成本效益（單位：查詢/秒/元）衡量單位成本下每秒完成的查詢數，計算方式為：首先計算吞吐量（throughput），即每秒完成的查詢數，假設處理  $I$  個查詢，吞吐量為  $I/T$ （單位：查詢/秒）；隨後，將吞吐量除以該查詢的執行成本，得到成本效益，其表示單一查詢的成本效益為其執行時間與成本的倒數，反映單位成本下的處理效率。公式如下：

$$\begin{aligned} Cost\ Efficiency\ (Single\ Query) &= \frac{Throughput}{Unit\ Query\ Cost} \\ &= \frac{1/T}{T \times VM\ Unit\ Price \times Presto\ Nodes} \\ &= \frac{1}{T^2 \times VM\ Unit\ Price \times Presto\ Nodes} \end{aligned}$$

● 公式四：總成本效益

針對多筆查詢，總成本效益（單位：查詢/秒/元）以整體吞吐量除以總執行成本計算。總吞吐量為查詢總數除以總執行時間；總執行成本由公式二得出。總成本效益公式如下，其中， $N$  為查詢總數，在本研究中為 99（筆）， $T_{total}$  為所有查詢的執行時間（單位：秒）加總。

$$\begin{aligned} Total\ Cost\ Efficiency &= \frac{Total\ Throughput}{Total\ Query\ Cost} \\ &= \frac{N/T_{total}}{Total\ Query\ Cost} \\ &= \frac{N}{T_{total} \times Total\ Query\ Cost} \end{aligned}$$

### 5-5-2 比較分析

首先，我們比較靜態配置以及本研究所提出的 Auto Scaling 機制其在成本效益的表現，並且分析其查詢執行時間、成本以及工作節點數量的關係，以評估兩種資源配置的方式在成本效益上的差異。如圖 26 中左下角成本效益的整體比較顯示，Auto

Scaling 在成本效益上具有顯著優勢，總成本效益提升將近 5%，主要得益於執行時間的縮短。雖然總成本增加 12%，但 Auto Scaling 透過增加 33% 的節點使用量，將查詢處理效率提升，進而在單位成本下實現更高的查詢吞吐量。這一結果表明，Auto Scaling 適合對成本效益敏感的場景，例如需要快速回應的大規模數據處理。然而，成本增幅高於查詢執行時間降幅的現象顯示著，Auto Scaling 的資源分配效率仍有優化空間，特別是在成本控制方面，需避免過度分配節點導致成本效益下降。

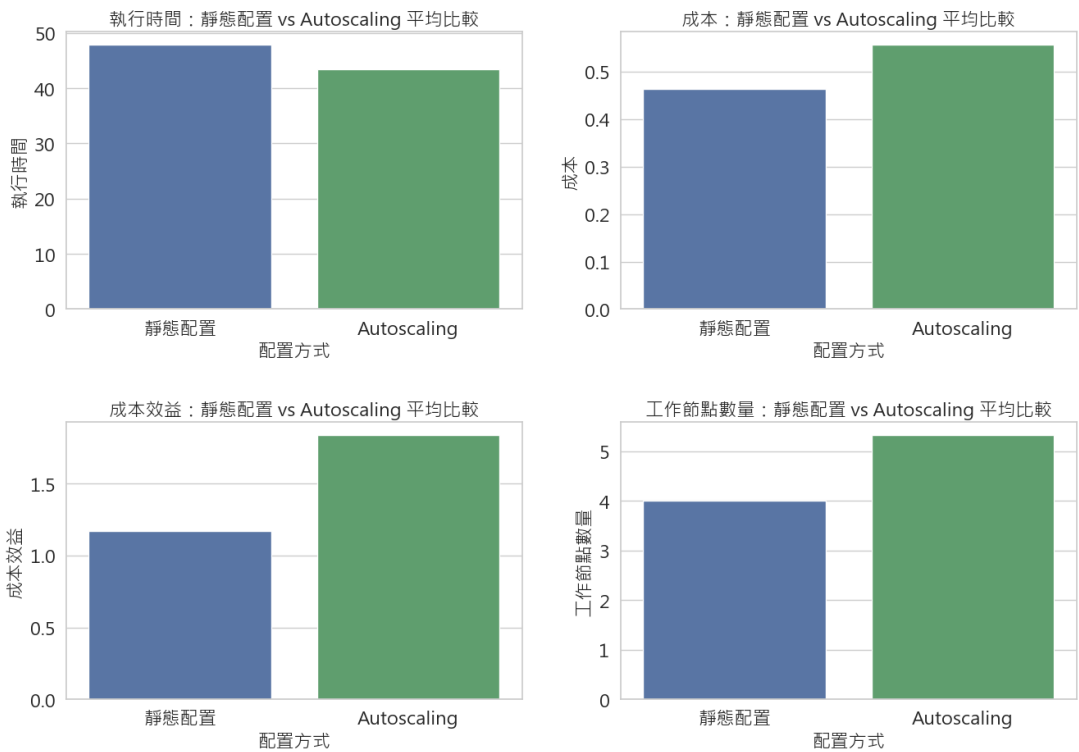


圖 26: 靜態配置與 Auto Scaling 綜合比較

為進一步探討我們分析 Auto Scaling 在成本效益上的影響分佈，並識別哪些查詢類型最適合或最不适合 Auto Scaling。分析方法包括：首先，計算每筆查詢的成本效益差異（Auto Scaling 成本效益減去靜態配置成本效益）；其次，對差異值進行排序，提取前 10 名與後 10 名的查詢進行分析比較。如圖 27 成本效益差異分佈分析所示，Auto Scaling 在成本效益上的兩極化表現。子查詢與連接查詢（如 Query41 和 Query84）在 Auto Scaling 下有顯著的成本效益提升，顯示 Auto Scaling 能有效應對這類查詢的資源需求，透過增加工作節點（如從 3 個增至 5 個）縮短執行時間，並在部分案例中降低成本。然而，聚合查詢（如 Query22）與窗口函數查詢（如

Query20) 在 Auto Scaling 下成本效益下降，主要是因為執行時間與成本的雙重上升，表明這些查詢對節點數增加的敏感性較低，甚至可能因過度分配導致效率損失。這些發現強調了查詢類型對 Auto Scaling 成本效益影響的差異，將 Auto Scaling 應用於子查詢與連接查詢有著較佳的表現，而對聚合查詢則需優化資源分配策略，確保工作節點數量的增加能帶來相應的成本效益提升。

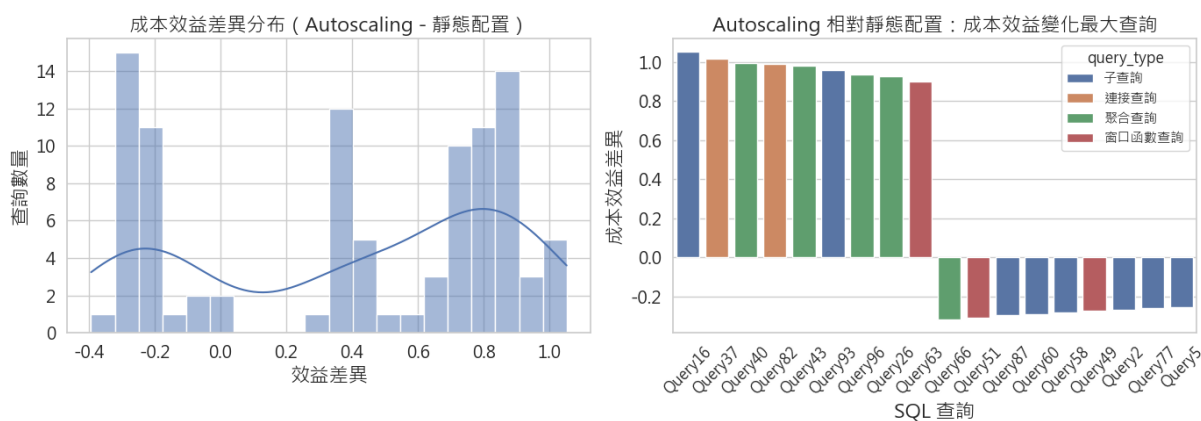


圖 27：成本效益分佈圖

### 5-5-3 成本效益與效能變化的關聯分析

為探討成本效益的變化（Auto Scaling 的指標減去靜態配置的指標）與執行時間、成本變化的關聯，分析哪些查詢類型在 Auto Scaling 下成本效益提升最多，並展示成本效益變化的驅動因素。分析方法包括：首先，計算每筆查詢的執行時間變化率（%）、成本變化率（%）與成本效益變化（%）；其次，排序成本效益提升與下降最多的前五名查詢，並分析其執行時間與成本的變化。

首先針對成本效益提升最多的前五名查詢進行分析，在表 21 中，五個查詢的平均執行時間減少了 47.36%，透過 Auto Scaling 將節點數量增加至 6 個（一個 Coordinator 與五個 Workers），提升了其數據處理的能力，尤其在子查詢、連接查詢與聚合查詢等運算較密集的场景中效率更佳。例如，子查詢 Query16 的執行時間縮短了 49%，連接查詢 Query84 與 Query30 則分別縮短了 47% 與 45%，除了執行時間的優化，查詢成本也出現平均 20.86% 的降幅，顯示 Auto Scaling 在提升效能的同時，透過此機制的資源分配，有效避免成本的增加，顯示在有限資源下達成更高輸出。

表 21：成本效益提升 Top 5 查詢及其效能變化

| 查詢      | 類型   | 成本效益變化 (%) | 查詢執行時間變化 (%) | 成本變化 (%) |
|---------|------|------------|--------------|----------|
| Query16 | 子查詢  | +105%      | -49%         | -23%     |
| Query91 | 聚合查詢 | +101%      | -47%         | -21%     |
| Query84 | 連接查詢 | +99%       | -47%         | -21%     |
| Query30 | 子查詢  | +99%       | -45%         | -18%     |
| Query81 | 子查詢  | +98%       | -46%         | -19%     |

接著對成本效益下降最多的前五名查詢進行分析，如表 22 所示，Query22 的成本效益下降 39.35%，其執行時間增加 21.45%，成本下降 8.91%，顯示 Auto Scaling 未能提升其查詢效率，因節點數量減少（從 4 降至 3）導致聚合查詢的並行處理能力不足，進而惡化成本效益。類似地，Query66、Query51、Query87 和 Query60 的成本效益下降（分別為 -31.68%、-31.00%、-29.42% 和 -28.85%）主要與成本的顯著上升（平均 +45.20%）有關，儘管執行時間略有減少（平均 -2.45%）。這表明 Auto Scaling 在這些查詢中會因為過度分配節點（多數從 4 增至 6）導致成本增加，未能有效提升查詢效率。

表 22：成本效益下降 Top 5 查詢及其效能變化

| 查詢      | 類型     | 成本效益變化 (%) | 查詢執行時間變化 (%) | 成本變化 (%) |
|---------|--------|------------|--------------|----------|
| Query22 | 聚合查詢   | -39%       | +21%         | -8%      |
| Query66 | 聚合查詢   | -31%       | -2%          | +16%     |
| Query51 | 窗口函數查詢 | -31%       | -2%          | +9%      |
| Query87 | 子查詢    | -29%       | -3%          | +44%     |
| Query60 | 子查詢    | -28%       | -4%          | +43%     |

綜合上述對於成本效益與效能變化的關聯分析，Auto Scaling 機制在處理運算密集型查詢時，對於提升查詢效能與成本效益具顯著效果。針對成本效益提升幅度最高的前五個查詢，當 Presto 節點 Scale-up 至六個節點（含一個 Coordinator 與 五個 Workers）時，其平均查詢執行時間減少達 47%，同時執行成本亦下降 20%。此一效能提升主要歸因於資源即時擴充所帶來的高併發處理能力，有效縮短查詢的執行時



間，對於子查詢、連接查詢與聚合查詢等運算需求較高的查詢類型效果較佳。然而，Auto Scaling 並非在所有查詢場景皆能實現正向成本效益。以成本效益下降最顯著的 Query22 為例，節點數量由四個減至三個，導致執行時間增加 21%，雖然成本略降 8%，但整體效益下降 39%。此外，Query66、Query51、Query87 與 Query60 雖在節點 Scale-up 後，執行時間略有改善，執行成本上升，使得成本效益呈現負成長。

#### 5-5-4 成本效益觀察

此子章節針對 TPC-DS 的 99 個 SQL 查詢的實驗結果進行分析評估靜態配置與本研究提出的 Auto Scaling 機制在成本效益方面的表現差異。根據實驗結果顯示，Auto Scaling 在多數情況下展現出優於靜態配置的成本效益表現。整體分析結果顯示，全部的查詢中，有 68 個查詢（約 68%）在採用 Auto Scaling 進行資源的配置後獲得成本效益上的改善，其平均提升率為 43%，中位數為 46%。雖然配對 t 檢定結果顯示未達統計顯著水準（ $p=0.168$ ，需小於 0.05），但整體效益上的提昇仍具有實務上的意義。分群分析進一步展示，不同查詢類型對 Auto Scaling 的適應性存在差異。以窗口函數與聚合查詢為例，成本效益提升的查詢比例分別為 80% 與 82%，且兩者均達統計顯著水準（ $p$  值分別為 0.018 與 0.001）。相對而言，子查詢類型的提升成本效益的比例為 56%，則未達統計顯著。此外，雖然連接查詢的樣本數僅為三筆，但在本實驗中皆出現正向改善，顯示潛在效益值得進一步研究。在《5-5-3 成本效益與效能變化的關聯分析》中則進一步證實，執行時間縮短是成本效益提升的關鍵驅動因素，但過度分配節點可能導致成本上升抵消效率提升。

綜合上述分析，Auto Scaling 機制能有效提升在多數查詢場景下的資源利用效率與成本效益，尤其在運算密集或具有明確階段性負載特徵的查詢類型中成效明顯。未來可以在相關的查詢應用系統部署與查詢調度設計上，納入查詢特性辨識與資源需求預測，以利 Auto Scaling 機制的最大效益。

## 六、結論及未來研究方向

本研究針對 Presto 靜態配置在不同查詢負載下的效能與資源利用問題，提出了一種基於 SQL 查詢負載評估的 Auto Scaling 機制。該機制整合機器學習（ML）預測模型、基礎設施即代碼（Infrastructure as Code）資源調整技術，以及 Apache Airflow 的 DAG 工作流程，實現 Presto Worker 節點的動態調整。通過在 Oracle Cloud Infrastructure（OCI）VM 環境中搭建 Presto 叢集以 TPC-DS 數據集進行測試，研究驗證了該機制的有效性。實驗結果顯示，Auto Scaling 將 99 條查詢的總執行時間縮減約 14%，並使大部分查詢延遲降低，其中聚合查詢的執行時間減少尤為顯著，顯示其在提升效能上的潛力。這一改善得益於動態資源分配，大多數查詢因負載增加而配置更多 Worker 節點，提升了並行處理能力。然而，同時觀察到並行開銷增加了 CPU Time，反映了動態調整的運算負擔。在成本效益方面，Auto Scaling 相較於靜態配置 y 總體提升約 5%，其得益於查詢執行時間的顯著縮短，儘管整體成本因節點數量增加約 33% 而上升約 12%。部分查詢實現了成本節省，尤其在資源密集型查詢中，顯示 Auto Scaling 在成本效益上的優勢。然而，成本效益的提升並非一致，部分查詢的成本效益因執行時間與成本的雙重上升導致下降，提示資源分配策略需針對查詢特性進行優化。此外，Auto Scaling 造成大部分查詢的記憶體使用增加，平均上升約 15-20%，主要由工作節點間資料交換的開銷引起，顯示節點數量提升的代價。

基於研究發現，本研究提出以下應用建議，以最大化 Auto Scaling 的實用性與效益。首先，應根據查詢類型差異化配置資源，針對延遲敏感型查詢積極採用 Auto Scaling，而對於資源密集型查詢，建議設置節點數上限，防止過度分配影響性價比。其次，開發實時成本效益監控系統，根據查詢負載動態調整節點數，確保成本增幅不超過執行時間降幅帶來的效益。最後，針對記憶體使用增加的問題，建議優化查詢計畫或採用資料壓縮技術，減少節點間資料交換量，平衡效能與資源消耗。

儘管 Auto Scaling 在查詢執行效率與成本效益上展現顯著優勢，但仍存在改進空間。未來研究可從以下兩個方向展開：

1. 針對查詢記憶體使用增加的問題，透過最佳化查詢計畫以減少 Shuffle 操作，或採用資料壓縮技術降低交換量，從而減少工作節點間資料交換的開銷。
2. 對於整體雲端成本的增加，透過設定成本閾值或查詢優先級，並開發多目標優化模型，來動態平衡延遲、資源和成本指標。

## 七、參考文獻

- [1] IDC, "Revelations in the Global DataSphere , 2024: Key Trends and Takeaways," 2024.  
<https://my.idc.com/getdoc.jsp?containerId=prCHC52667624> (Accessed 12, 2024)
- [2] Amazon Web Services, Inc, "Cloud Computing Concepts Hub,"  
[https://aws.amazon.com/what-is/presto/?nc1=h\\_ls](https://aws.amazon.com/what-is/presto/?nc1=h_ls) (Accessed 12, 2024)
- [3] The Presto Foundation. "Presto Official Website" <https://prestodb.io/> (Accessed 12, 2024)
- [4] The Apache Software Foundation. "Apache Airflow Official Website"  
<https://airflow.apache.org/> (Accessed 12, 2024)
- [5] Cloud Software Group, Inc. "TIBCO: What is massively parallel processing".  
<https://www.tibco.com/> (Accessed 12, 2024)
- [6] HashiCorp. "Terraform Official Documentation."  
<https://developer.hashicorp.com/terraform> (Accessed 12, 2024)
- [7] Sethi, R., et al. (2019). "Presto: Sql on everything," in 2019 IEEE 35th International Conference on Data Engineering (ICDE) (Macao: IEEE), pp. 1802–1813.
- [8] S. Gourishetti, "Performance Optimization in Distributed SQL Environments: A Comprehensive Analysis of Presto Query Engine," International Journal of Scientific Research in Computer Science Engineering and Information Technology, vol. 10, no. 6, pp. 241-253, November 2024.
- [9] P. Vemasani, Sai M. Vuppalapati, "Achieving Agility through Auto-Scaling: Strategies for Dynamic Resource Allocation in Cloud Computing," International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol. 12, no. 4, pp. 3169–3177, Apr. 2024.
- [10] S. Alharthi, et al., "Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions," Sensors, vol. 24, no. 17, p. 5551, Aug. 2024.
- [11] C. Tang, et al., "Forecasting SQL Query Cost at Twitter," in Proc. 2021 IEEE International Conference on Cloud Engineering (IC2E), 2021, pp. 154-160.

[12] Oracle. "Cloud Cost Estimator." <https://www.oracle.com/tw/cloud/costestimator.html> (Accessed 12, 2024)

[13] TPC. "TPC BENCHMARK <sup>TM</sup> DS." [https://www.tpc.org/tpc\\_documents\\_current\\_versions/current\\_specifications5.asp](https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp) (Accessed 12, 2024)

[14] The Presto Foundation. "Presto Concepts Overview." <https://prestodb.io/docs/0.291/overview/concepts.html#overview>

[15] Oracle. "Oracle Cloud Infrastructure Documentation." <https://docs.oracle.com/en-us/iaas/Content/dev/terraform/tutorials/tf-simple-infrastructure.htm>

[16] The Presto Foundation. "Python Client | Presto document." <https://prestodb.io/docs/current/clients/python.html>

[17] E. W. Davis, "Application of the Massively Parallel Processor to Database Management Systems," in Proceedings of the National Computer Conference, May 16-19, 1983. pp. 299-307.